CS-GY 6923: Lecture 4
Continue on Bayesian Perspective, Modeling Language

NYU Tandon School of Engineering, Akbar Rafiey

Course news

- First written problem set due next Oct 02.
- Lab 3 is released, due on Oct 08.

Recap: probabilistic modeling

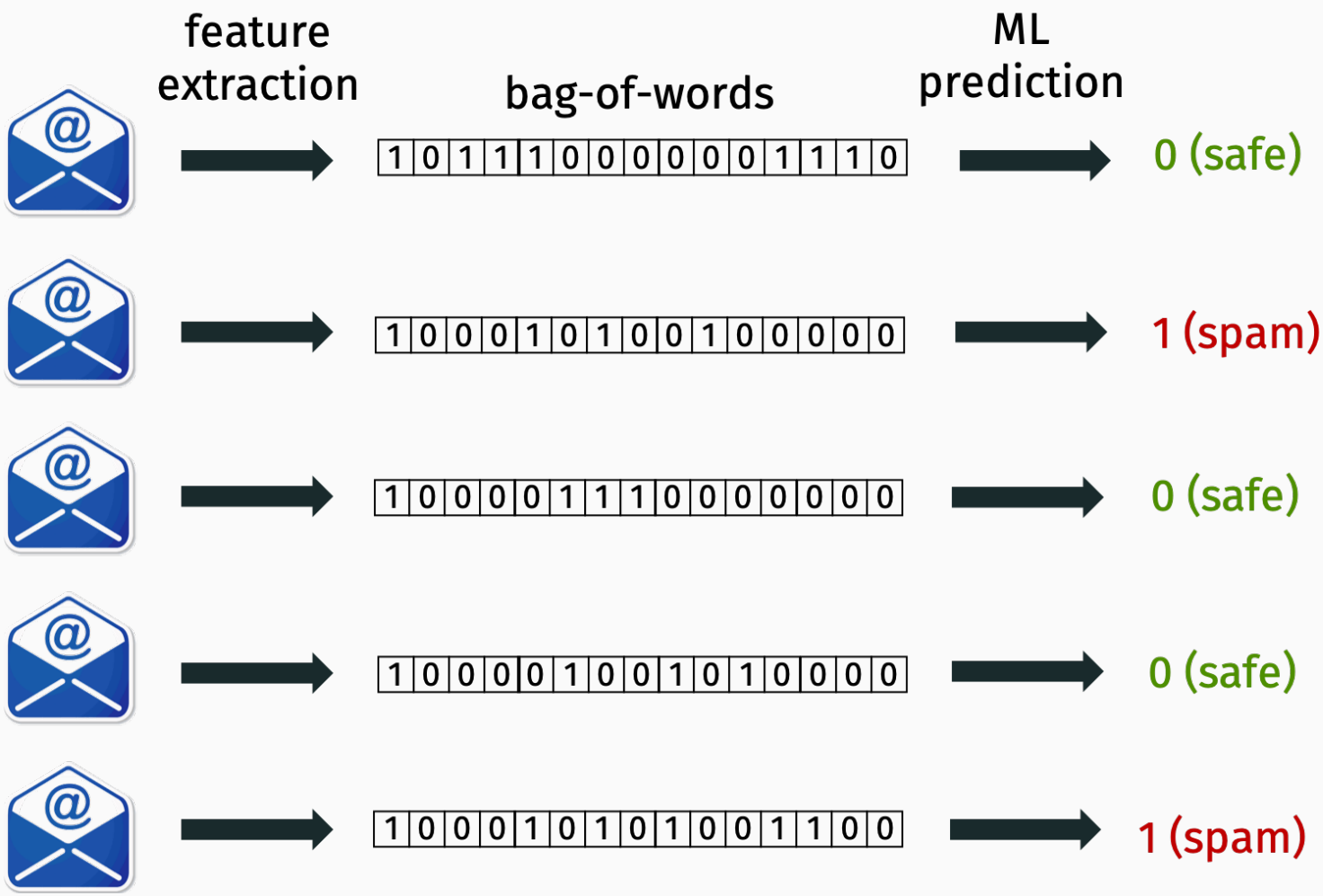
In a Bayesian or Probabilistic approach to machine learning we always start by conjecturing a

probabilistic model

that plausibly could have generated our data.

- The model guides how we make predictions.
- The model typically has unknown parameters $\vec{\beta}$ and we try to find the most reasonable parameters based on observed data.

Recap: spam prediction



Recap: probabilistic model for email

Probabilistic model for (bag-of-words, label) pair $(\mathbf{x}, y)$:

- Set $y = 0$ (not spam) with probability p_0 and $y = 1$ (spam) with probability $p_1 = 1 - p_0$.
  - p_0 is probability an email is not spam (e.g. 99%).
  - p_1 is probability an email is spam (e.g. 1%).
- If $y = 0$, for each i, set $x_i = 1$ with prob. p_{i0}.
- If $y = 1$, for each i, set $x_i = 1$ with prob. p_{i1}.

Unknown model parameters:

- p_0, p_1,
- $p_{10}, p_{20}, \ldots p_{d0}$, one for each of the d vocabulary words.
- $p_{11}, p_{21}, \ldots p_{d1}$, one for each of the d vocabulary words.

Recap: parameter estimation

Reasonable way to set parameters:

- Set p_0 and p_1 to the empirical fraction of not spam/spam emails.
- For each word i, set p_{i0} to the empirical probability word i appears in a non-spam email.
- For each word i, set p_{i1} to the empirical probability word i appears in a spam email.

Done with modeling

on to prediction

Classification rule

Given unlabeled input $(\mathbf{w}, \_\_\_)$, choose the label $y \in \{0, 1\}$ which is most likely given the data. Recall $\mathbf{w} = [0, 0, 1, \ldots, 1, 0]$.

Classification rule: maximum a posterior (MAP) estimate.

Step 1. Compute:

- $p(y = 0 \mid \mathbf{w})$: prob. $y = 0$ given observed data vector $\mathbf{w}$.
- $p(y = 1 \mid \mathbf{w})$: prob. $y = 1$ given observed data vector $\mathbf{w}$.

Step 2. Output: 0 or 1 depending on which probability is larger.

$p(y = 0 \mid \mathbf{w})$ and $p(y = 1 \mid \mathbf{w})$ are called **posterior** probabilities.

Evaluating the posterior

How to compute the posterior? **Bayes rule!**

$$p(y = 0 \mid \mathbf{w}) = \frac{p(\mathbf{w} \mid y = 0)p(y = 0)}{p(\mathbf{w})} \tag{1}$$

$$\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}} \tag{2}$$

- **Prior:** Probability in class 0 prior to seeing any data.
- **Posterior:** Probability in class 0 after seeing the data.

Evaluating the posterior

Goal is to determine which is larger:

$$p(y = 0 \mid \mathbf{w}) = \frac{p(\mathbf{w} \mid y = 0)p(y = 0)}{p(\mathbf{w})} \qquad \text{vs.}$$

$$p(y = 1 \mid \mathbf{w}) = \frac{p(\mathbf{w} \mid y = 1)p(y = 1)}{p(\mathbf{w})}$$

- We can ignore the evidence $p(\mathbf{w})$ since it is the same for both sides!
- $p(y = 0)$ and $p(y = 1)$ already known (computed from training data). These are our computed parameters p_0, p_1.
- $p(\mathbf{w} \mid y = 0) = ?$ $p(\mathbf{w} \mid y = 1) = ?$

Evaluating the posterior

Consider the example $\mathbf{w} = [0, 1, 1, 0, 0, 0, 1, 0]$.

Recall that, under our model, index i is 1 with probability p_{i0} if we are not spam, and 1 with probability p_{i1} if we are spam .

$$p(\mathbf{w} \mid y = 0) =$$

$$p(\mathbf{w} \mid y = 1) =$$

Naive bayes classifier

Training/Modeling: Use existing data to compute:

- $p_0 = p(y = 0), p_1 = p(y = 1)$
- For all i compute:
  - $p_{i0} = p(w_i = 1 \mid y = 0)$ and $(1 - p_{i0}) = p(w_i = 0 \mid y = 0)$
  - $p_{i1} = p(w_i = 1 \mid y = 1)$ and $(1 - p_{i1}) = p(w_i = 0 \mid y = 1)$

Prediction:

- For new input $\mathbf{w}$:
  - Compute $p(\mathbf{w} \mid y = 0) = \prod_i p(w_i \mid y = 0)$
  - Compute $p(\mathbf{w} \mid y = 1) = \prod_i p(w_i \mid y = 1)$
- Return $y = 0$ if
  $p(\mathbf{w} \mid y = 0) \cdot p(y = 0) \geq p(\mathbf{w} \mid y = 1) \cdot p(y = 1)$
- Return $y = 1$ otherwise

Other applications of
the bayesian perspective on regression

Bayesian regression

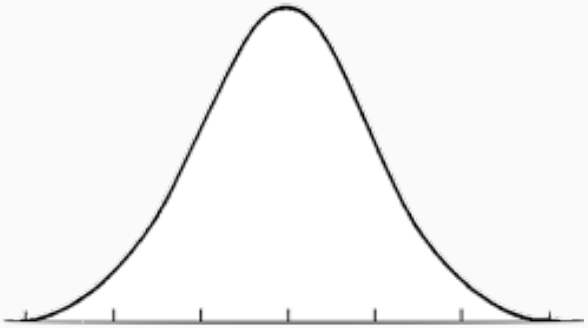
The Bayesian view offers an interesting alternative perspective on many machine learning techniques.

Example: Linear Regression.

Probabilistic model:

$$y = \langle \mathbf{x}, \boldsymbol{\beta} \rangle + \eta$$

where the η drawn from $N(0, \sigma^2)$ is **random Gaussian noise**.



$$Pr(\eta = z) \sim$$

The symbol $\sim$ means "is proportional to".

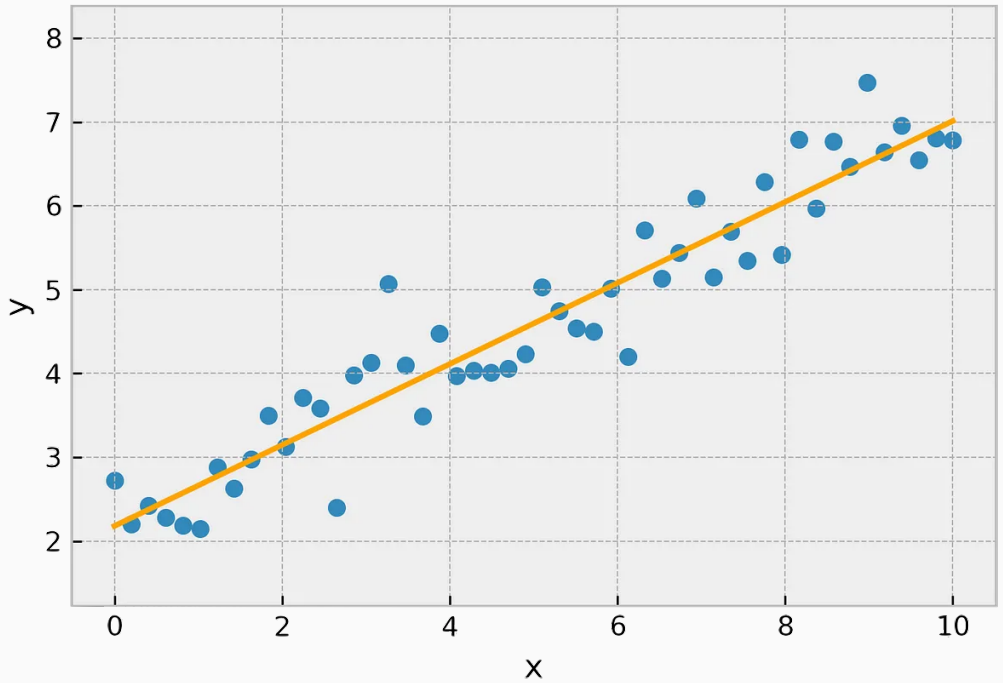
Illustration

Example: Linear Regression.

Probabilistic model:

$$y = \langle \mathbf{x}, \boldsymbol{\beta} \rangle + \eta$$

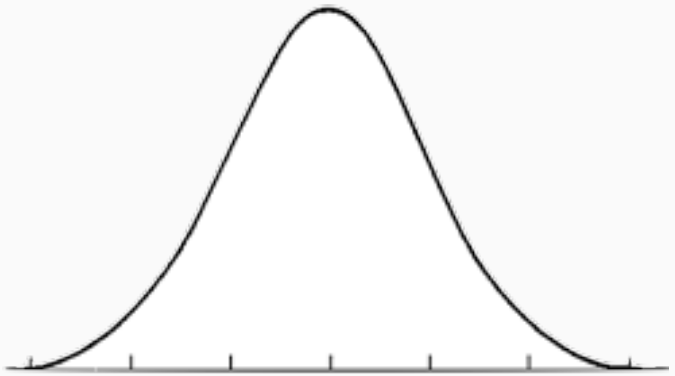
where the η drawn from $N(0, \sigma^2)$ is **random Gaussian noise**. The noise is independent for different inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n$.



Gaussian distribution refresher

Names for same thing: Normal distribution, Gaussian distribution, bell curve.

Parameterized by mean μ and variance σ^2.

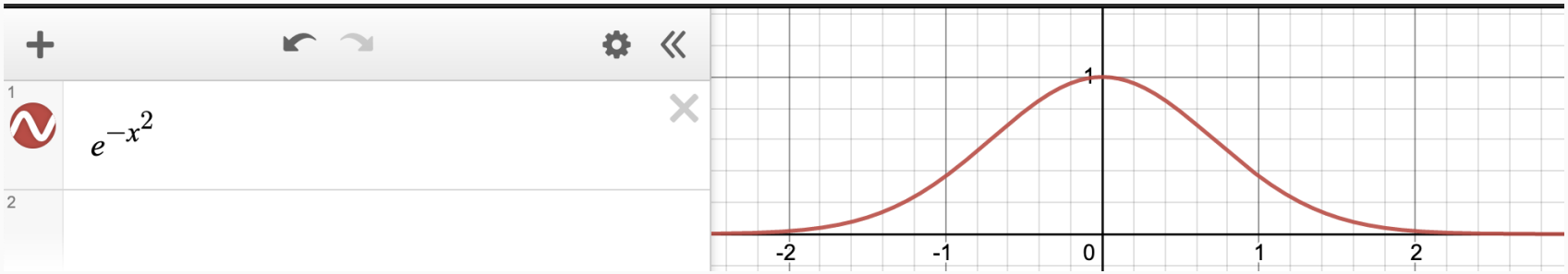


η is a continuous random variable, so it has a probability density function $p(\eta)$ with $\int_{-\infty}^{\infty} p(\eta)d\eta = 1$

$$p(\eta) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{\eta-\mu}{\sigma})^2}$$

Gaussian distribution refresher

The important thing to remember is that the the PDF falls off exponentially as we move further from the mean.



The normalizing constant in front 1/2, etc. don't matter so much.

Bayesian regression

How should we select β for our model?

Also use a Bayesian approach!

First thought: choose β to maximize:

$$\text{posterior} = \Pr(\beta \mid \mathbf{X}, \mathbf{y}) = \frac{\Pr(\mathbf{X}, \mathbf{y} \mid \beta)\Pr(\beta)}{\Pr(\mathbf{X}, \mathbf{y})} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}.$$

But in this case, we don't have a prior – no values of β are inherently more likely than others.

Choose β to maximize just the likelihood:

$$\frac{\Pr(\mathbf{X}, \mathbf{y} \mid \beta)\cancel{\Pr(\beta)}}{\cancel{\Pr(\mathbf{X}, \mathbf{y})}} = \frac{\text{likelihood} \times \cancel{\text{prior}}}{\cancel{\text{evidence}}}.$$

This is called the **maximum likelihood estimate**.

Fixed design linear regression

- Often we think of $\mathbf{X}$ as fixed and deterministic, and only $\mathbf{y}$ is generated at random in the model. This is called the fixed design setting.
- We can also consider a randomized design setting, but it is slightly more complicated.
- In the fixed design setting our task of maximizing $\Pr(\mathbf{X}, \mathbf{y} \mid \beta)$ simplifies to maximizing

$$\max_{\beta} \Pr(\mathbf{y} \mid \beta)$$

Maximum likelihood estimate

Data:

$$\mathbf{X} = \begin{bmatrix} — & \mathbf{x}_1 & — \\ — & \mathbf{x}_2 & — \\ & \vdots & \\ — & \mathbf{x}_n & — \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Model: $y_i = \langle \mathbf{x}_i, \boldsymbol{\beta} \rangle + \eta_i$ where $p(\eta_i = z) \sim e^{-z^2/2\sigma^2}$ and $\eta_1, \ldots, \eta_n$ are independent.

Exercise:

$$\Pr(\mathbf{y} \mid \boldsymbol{\beta}) \sim$$

Log likelihood

Easier to work with the **log likelihood**:

$$\begin{aligned}
\arg\max_{\beta} \Pr(\mathbf{X}, \mathbf{y} \mid \beta) &= \arg\max_{\beta} \prod_{i=1}^{n} e^{-(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 / 2\sigma^2} \\
&= \arg\max_{\beta} \ \log \left(\prod_{i=1}^{n} e^{-(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 / 2\sigma^2} \right) \\
&= \arg\max_{\beta} \sum_{i=1}^{n} -(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 / 2\sigma^2 \\
&= \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \langle \mathbf{x}_i, \beta \rangle)^2 .
\end{aligned}$$

Maximum likelihood estimator

Conclusion: Choose β to minimize:

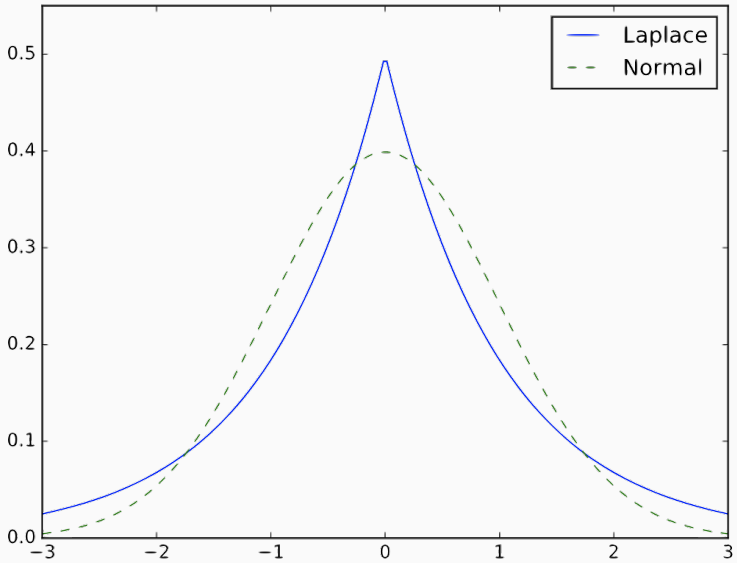
$$\sum_{i=1}^{n}(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 = \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

This is a completely different justification for squared loss!

Minimizing the ℓ_2 loss is "optimal" when you assume your data follows a linear model with i.i.d. Gaussian noise.

Bayesian regression

If we had modeled our noise η as Laplace noise, we would have found that minimizing $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_1$ was optimal.



$$Pr(\eta = z) \sim$$

Laplace noise has "heavier tails", meaning that it results in more outliers.

This is a completely different justification for ℓ_1 loss.

Bayesian regularization

- We can add another layer of probabilistic modeling by also assuming $\boldsymbol{\beta}$ is random and comes from some distribution, which encodes our prior belief on what the parameters are.
- Recall Maximum a posteriori (MAP estimation):

$$\Pr(\boldsymbol{\beta} \mid \mathbf{X}, \mathbf{y}) = \frac{\Pr(\mathbf{X}, \mathbf{y} \mid \boldsymbol{\beta}) \Pr(\boldsymbol{\beta})}{\Pr(\mathbf{X}, \mathbf{y})}.$$

- Assume values in $\boldsymbol{\beta} = [\beta_1, \ldots, \beta_d]$ come from some distribution.
- What is a common model?

Bayesian regularization

- **Common model:** Each β_i drawn from $N(0, \gamma^2)$, i.e. normally distributed, independent.
- Encodes a belief that we are unlikely to see models with very large coefficients.

Goal: choose $\boldsymbol{\beta}$ to maximize:

$$\Pr(\boldsymbol{\beta} \mid \mathbf{X}, \mathbf{y}) = \frac{\Pr(\mathbf{X}, \mathbf{y} \mid \boldsymbol{\beta}) \Pr(\boldsymbol{\beta})}{\Pr(\mathbf{X}, \mathbf{y})}.$$

Bayesian regularization

Goal: choose $\boldsymbol{\beta}$ to maximize:

$$\Pr(\boldsymbol{\beta} \mid \mathbf{X}, \mathbf{y}) = \frac{\Pr(\mathbf{X}, \mathbf{y} \mid \boldsymbol{\beta}) \Pr(\boldsymbol{\beta})}{\Pr(\mathbf{X}, \mathbf{y})}.$$

- We can still ignore the "evidence" term $\Pr(\mathbf{X}, \mathbf{y})$ since it is a constant that does not depend on $\boldsymbol{\beta}$.
- $\Pr(\boldsymbol{\beta}) = \Pr(\beta_1) \cdot \Pr(\beta_2) \cdot \ldots \cdot \Pr(\beta_d)$
- If each β_i drawn from $N(0, \gamma^2)$, $\Pr(\boldsymbol{\beta}) \sim$?

Bayesian regularization

Easier to work with the **log likelihood**:

$$\begin{aligned}
&\arg\max_{\beta} \Pr(\mathbf{X}, \mathbf{y} \mid \beta) \cdot \Pr(\beta) \\
&= \arg\max_{\beta} \prod_{i=1}^{n} e^{-(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 / 2\sigma^2} \cdot \prod_{i=1}^{n} e^{-(\beta_i)^2 / 2\gamma^2} \\
&= \arg\max_{\beta} \sum_{i=1}^{n} -(y_i - \langle \mathbf{x}_i, \beta \rangle)^2 / 2\sigma^2 + \sum_{i=1}^{d} -(\beta_i)^2 / 2\gamma^2 \\
&= \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \langle \mathbf{x}_i, \beta \rangle)^2 + \frac{\sigma^2}{\gamma^2} \sum_{i=1}^{d} (\beta_i)^2
\end{aligned}$$

Choose β to minimize $\|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \frac{\sigma^2}{\gamma^2}\|\beta\|_2^2$.

Completely different justification for ridge regularization!

Bayesian regularization

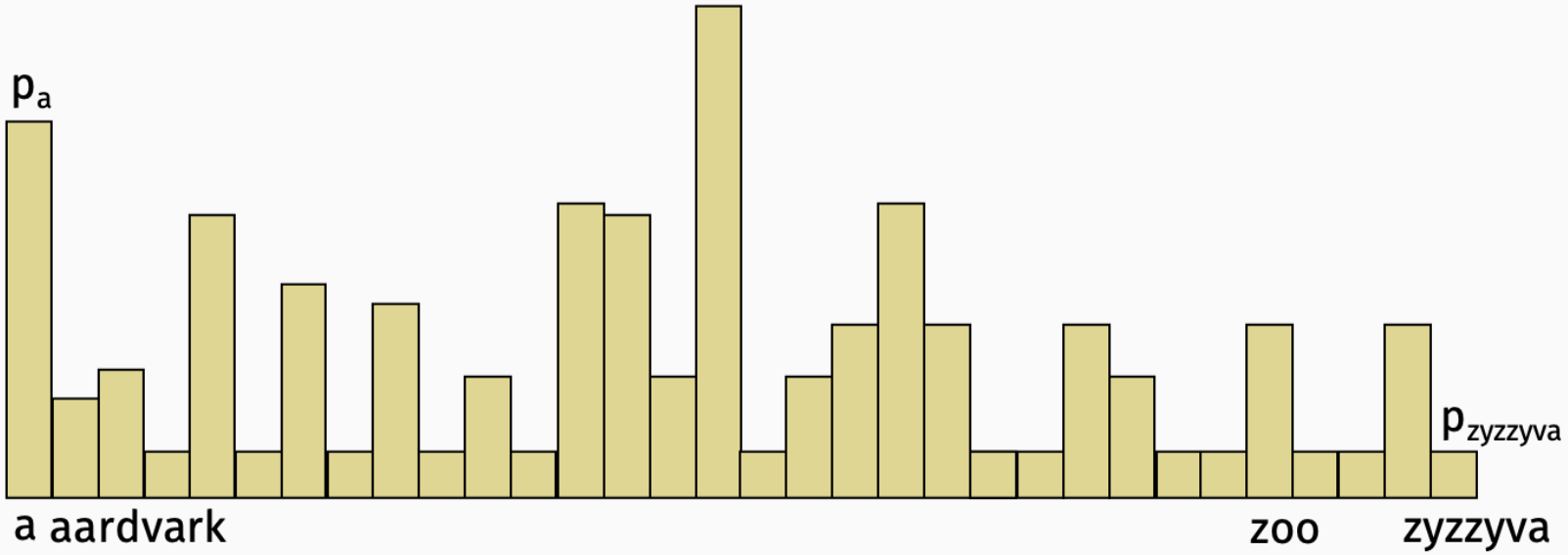
Test your intuition: What modeling assumption justifies LASSO regularization: $\min \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_1$?

Modeling language

Generative ML

Key idea behind generative ML:

- Build a very good probabilistic model for your data.
- Use that model to generate realistic looking new data.



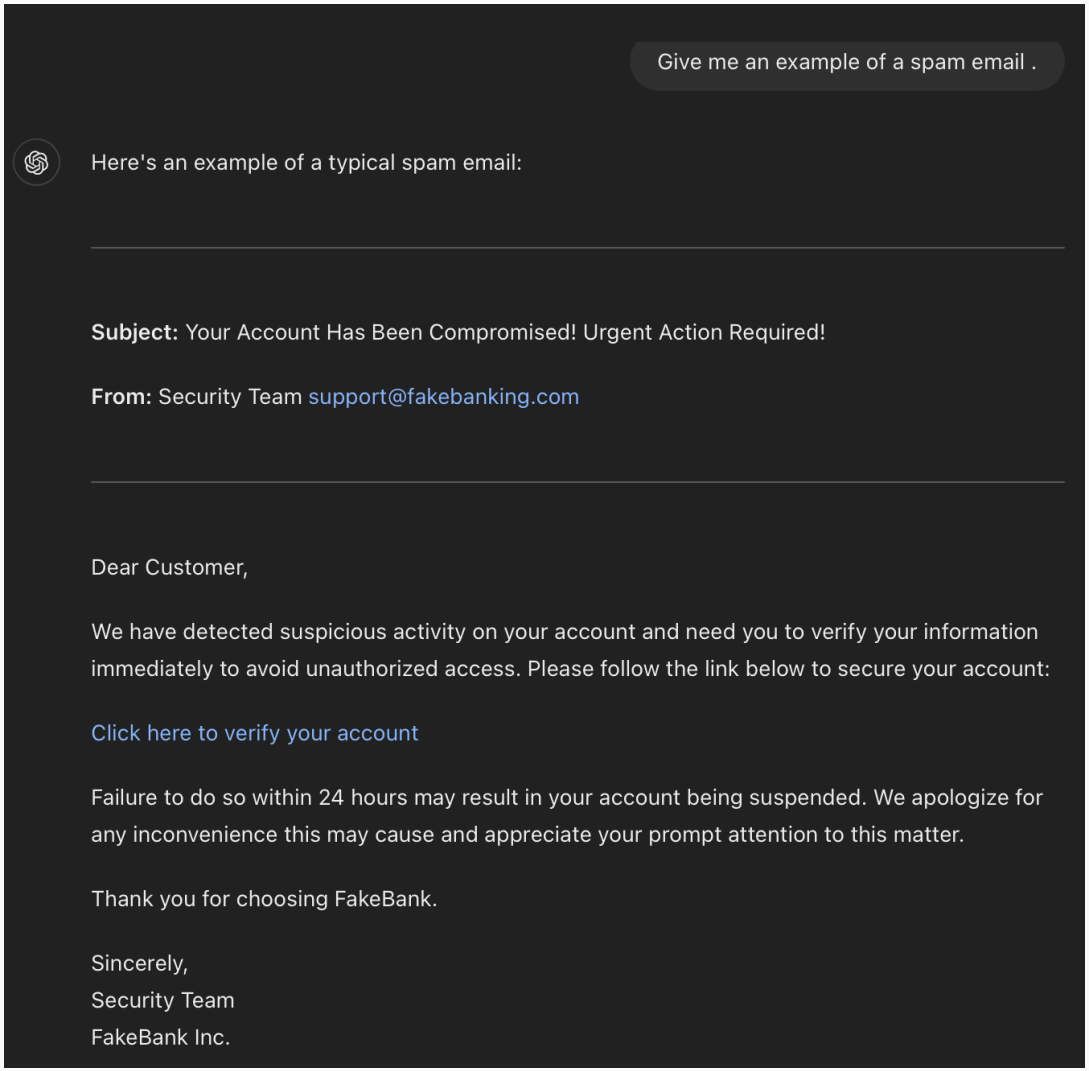
Example

Key idea behind generative ML: Build a very good probabilistic model for your data. Use that model to generate realistic looking new data.

Email example from our model: *Keeps retaining in astro associated to no garden superconducting whistleblower on effusion eigenvalue jobs worker for car shortlist villa depictions fitness the easter veto devices expressed secondary user metal this administrative the do of to struct coffee online cde the open through requirement stamps you job g thus drop stations.*

Language modeling

How do we go from this to something more like what modern models can produce?



Language modeling

How do we go from jumbled words to something more like what modern models can produce?

Dear Customer,

We have detected suspicious activity on your account and need you to verify your information immediately to avoid unauthorized access. Please follow the link below to secure your account:

Click here to verify your account

Failure to do so within 24 hours may result in your account being suspended. We apologize for any inconvenience this may cause and appreciate your prompt attention to this matter.

Thank you for choosing FakeBank.

Sincerely,
Security Team
FakeBank Inc.

Warning signs of spam:

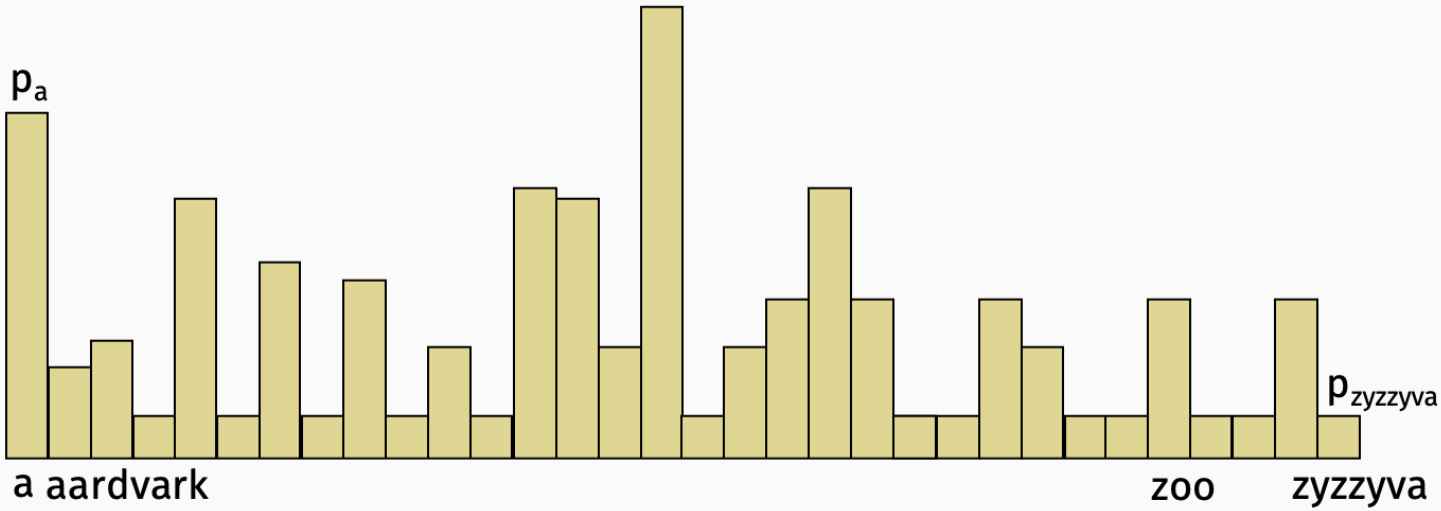
- **Urgency:** The message creates a sense of urgency ("urgent action required").
- **Unfamiliar sender:** The email address doesn't seem to match a legitimate institution.
- **Phishing link:** The link directs to a suspicious URL that doesn't belong to the claimed company.
- **Generic greeting:** It doesn't use your name, just "Dear Customer."
- **Threat:** It warns of a negative consequence if immediate action is not taken.

Always double-check such emails before clicking on any links or providing personal information!

Autoregressive models

Main issue: Our model lacks context!

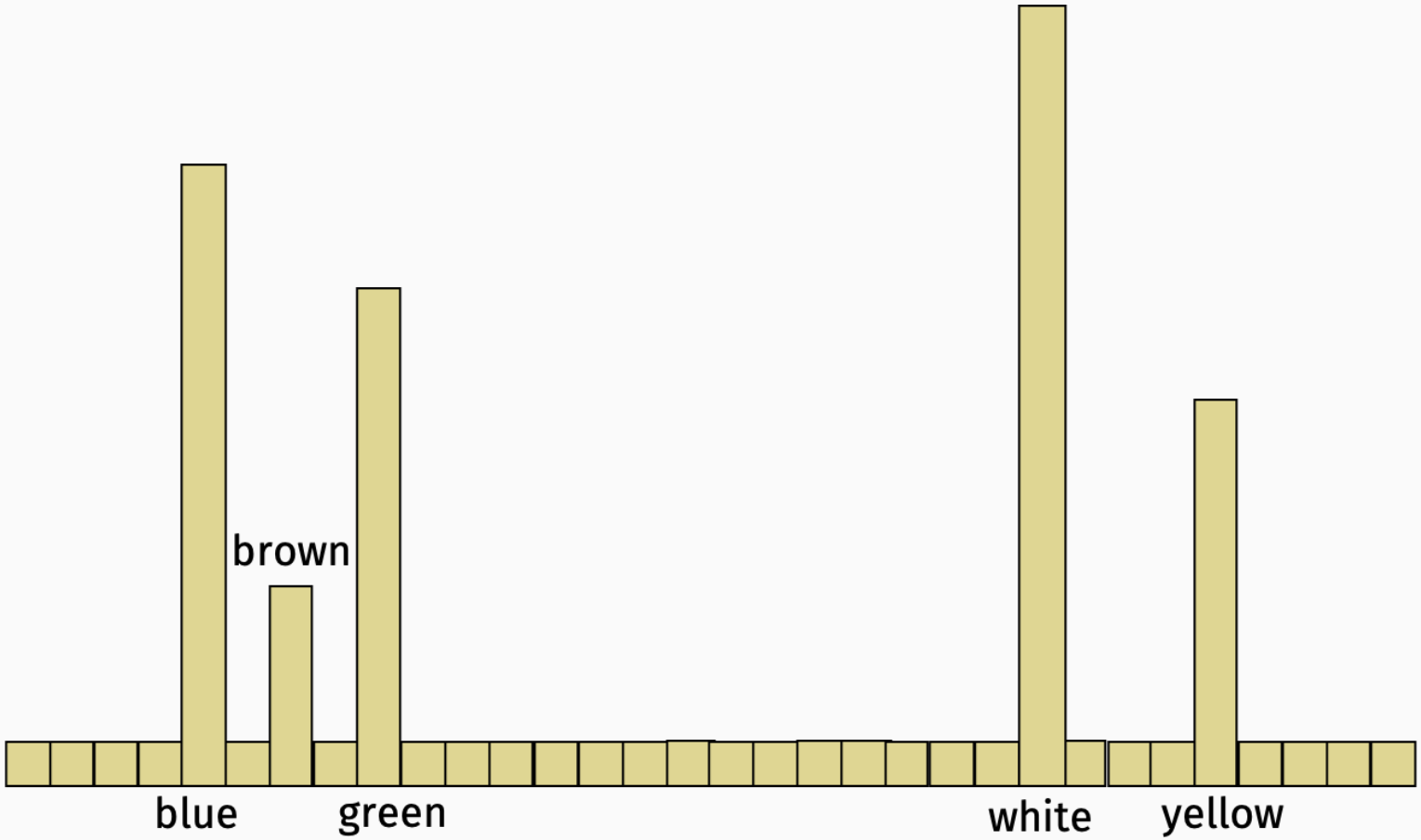
The color of the dress is _____.



Autoregressive models

Main issue: Our model lacks context!

The color of the dress is _____.



Autoregressive models

Key idea: Distribution that a word is chosen from should depend on previous words in the sentence/paragraph.

Consider generating a sentence with words $x_1, x_2, \ldots, x_n$.

- Initialize the first word x_1 of the sentence (e.g. at random).
- Choose x_2 based on x_1.
- Choose x_3 based on $x_1, x_2, \ldots$

Concretely, set $x_i = w$ with probability:

$$P(x_i = w \mid x_{i-1}, x_{i-2}, \ldots, x_1).$$

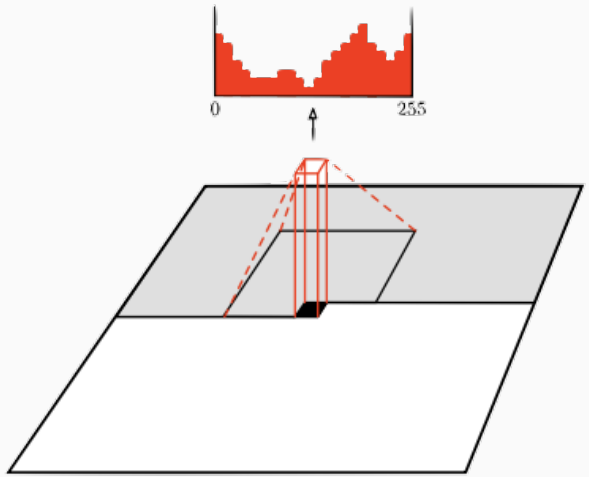
Autoregressive models

Autoregressive model's generate text in order.

- How most humans write sentences, emails, short text.
- How the latest modern language models write text (e.g. the GPT family of models.)

This is not the only approach to generative modeling, but it is one that works fairly well in practice, especially for text.

Can also be used e.g. for images, but no longer state of the art.



Autoregressive models

Can also be used e.g. for images, but no longer state of the art.

Pixel Recurrent Neural Networks

Aäron van den Oord AVDNOORD@GOOGLE.COM
Nal Kalchbrenner NALK@GOOGLE.COM
Koray Kavukcuoglu KORAYK@GOOGLE.COM

Google DeepMind

Abstract

Modeling the distribution of natural images is a landmark problem in unsupervised learning. This task requires an image model that is at once expressive, tractable and scalable. We present a deep neural network that sequentially predicts the pixels in an image along the two spatial dimensions. Our method models the discrete probability of the raw pixel values and en-

Figure 1. Image completions sampled from a PixelRNN.

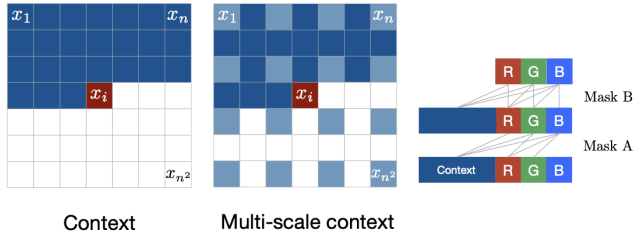


Figure 2. **Left**: To generate pixel x_i one conditions on all the previously generated pixels left and above of x_i. **Center**: To generate a pixel in the multi-scale case we can also condition on the subsampled image pixels (in light blue). **Right**: Diagram of the connectivity inside a masked convolution. In the first layer, each of the RGB channels is connected to previous channels and to the context, but is not connected to itself. In subsequent layers, the channels are also connected to themselves.

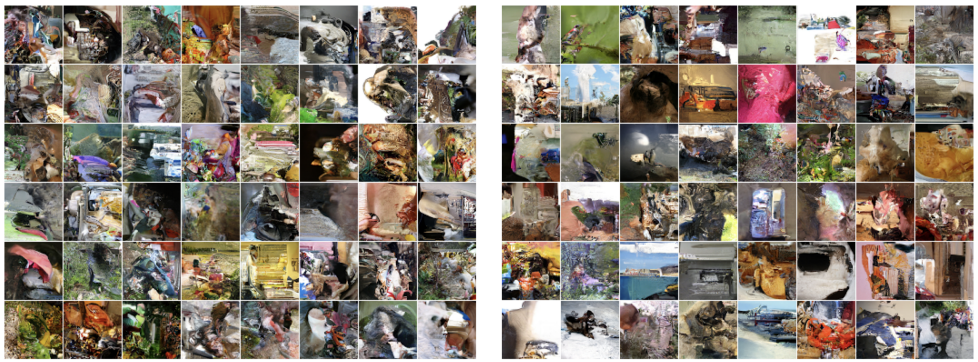
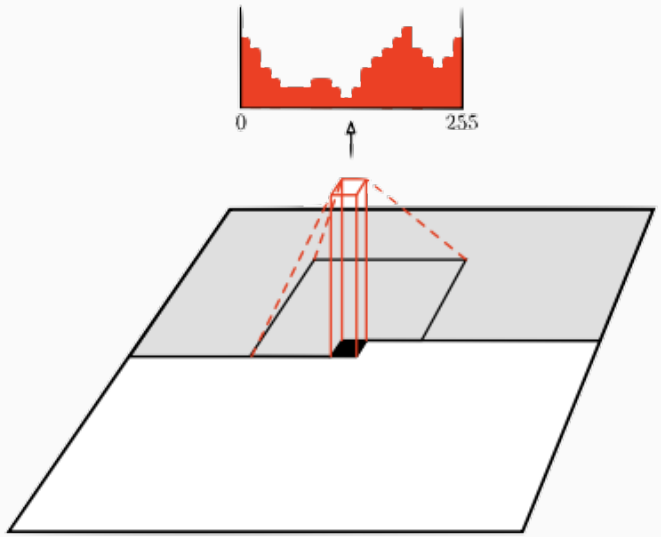


Figure 8. Samples from models trained on ImageNet 64x64 images. Left: normal model, right: multi-scale model. The single-scale model trained on 64x64 images is less able to capture global structure than the 32x32 model. The multi-scale model seems to resolve this problem. Although these models get similar performance in log-likelihood, the samples on the right do seem globally more coherent.

Autoregressive models

Can also be used e.g. for images, but no longer state of the art.



$$p(x_i) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots p(x_i \mid x_1, x_2, \dots, x_{i-1})$$

$$p(x_i; \beta) = \prod_{j=1}^{i} p(x_j \mid x_1, \dots, x_{j-1}; \beta) \quad \text{(parametrize a model by } \beta\text{)}$$

Autoregressive models

Can also be used e.g. for images, but no longer state of the art.

$$p(x_i) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots p(x_i \mid x_1, x_2, \dots, x_{i-1})$$

$$p(x_i; \beta) = \prod_{j=1}^{i} p(x_j \mid x_1, \dots, x_{j-1}; \beta) \quad \text{(parametrize a model by } \beta\text{)}$$

$$\log p(x_i, \beta) = \sum_{j=1}^{i} \log p(x_j \mid x_1, \dots, x_{j-1}; \beta) \qquad \text{(MLE)}$$

Different architectures to model $p(x_j \mid x_1, \dots, x_{j-1}; \beta)$:

- Suggestion?

Autoregressive models

Different architectures to model $p(x_j \mid x_1, \ldots, x_{j-1}; \beta)$:

- Convolutional Models (PixelCNN)
- Recurrent Models (PixelRNN)
- Transformer-based Models (ImageGPT, GPT)
- All of the above approaches need good (semantic) word embeddings. We will learn about this later in the course.
- Today: simple model and simple complexity

Limited lookback

Key idea: Distribution that a word is chosen from should depend on previous **k words** in the sentence/paragraph. k is a parameter that controls model complexity.

Limited lookback

Key idea: Distribution that a word is chosen from should depend on previous **k words** in the sentence/paragraph. k is a parameter that controls model complexity.

Consider generating a sentence with words $x_1, x_2, \ldots, x_n$.

- Initialize the first k word $x_1, \ldots, x_k$ of the sentence (e.g. at random).
- Choose x_{k+1} based on $x_1, \ldots, x_k$.
- Choose x_{k+2} based on $x_2, \ldots, x_{k+1}$.
- Choose x_{k+3} based on $x_3, \ldots, x_{k+2}$.
- $\ldots$

Set $x_i = w$ with probability:

$$P(x_i = w \mid x_{i-1}, x_{i-2}, \ldots, x_{i-k}).$$

Probabilistic model

Set $x_i = w$ with probability:

$$P(x_i = w \mid x_{i-k}, \ldots, x_{i-2}, x_{i-1}).$$

This probability can be tractably estimate from our data!

It is exactly the same as the probability of observing the $k+1$-gram $[x_{i-k}, \ldots, x_{i-2}, x_{i-1}, w]$.

Training:

- For corpus of text, collect all $k+1$-grams and record their frequency. (not much optimization involved!)

Prediction:

- At step i, sample from the subset of $k+1$ grams starting with $[x_{i-k}, \ldots, x_{i-2}, x_{i-1}]$, with probability proportional to their frequency.

Example

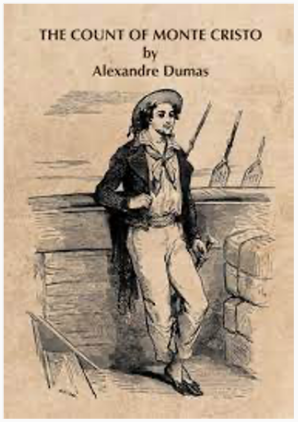
The color of the dress is _____.

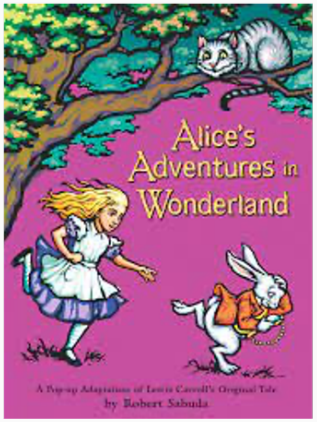
- Reasonable completions for $k = 2$:

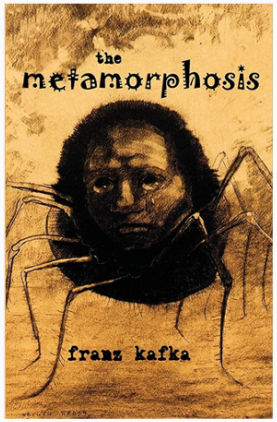
- Reasonable completions for $k = 5$:

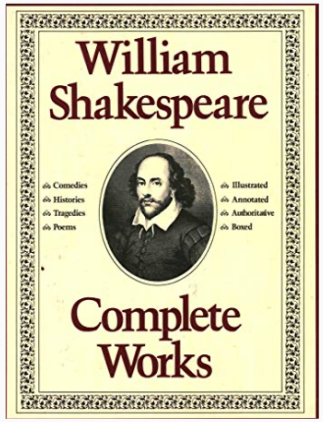
Lab 3[1]

- Train model on free books from Project Gutenberg.







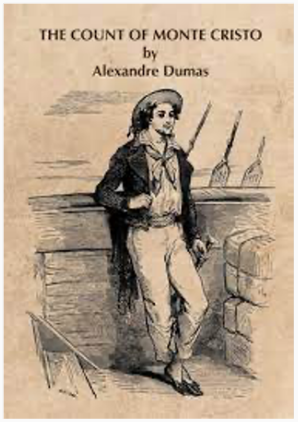


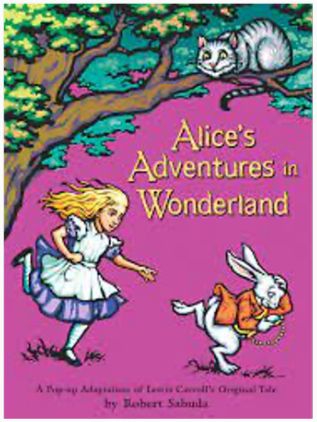
- Evaluate effect of changing k. Tradeoff between better performance and more "copying" from the course text.

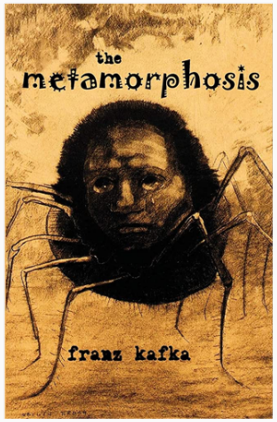
[1] Credit to Raphael Meyer

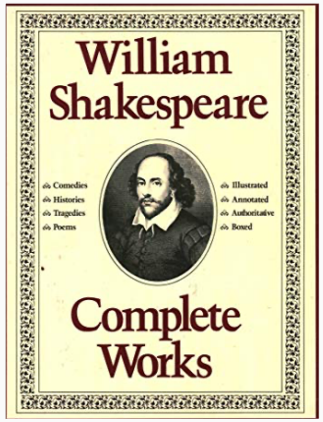
Lab 3

- Train model on free books from Project Gutenberg.







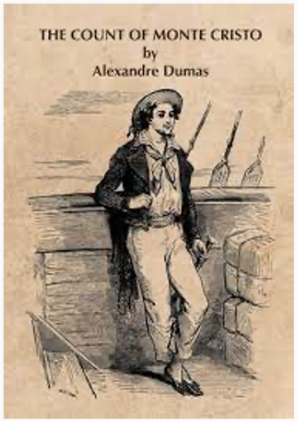


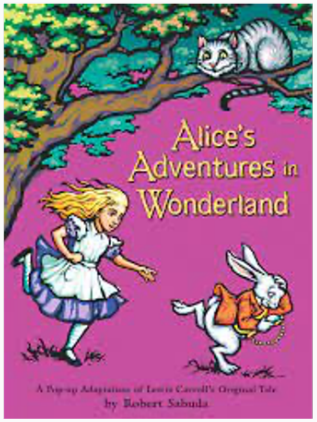
- Evaluate effect of changing k. Tradeoff between better performance and more "copying" from the course text.

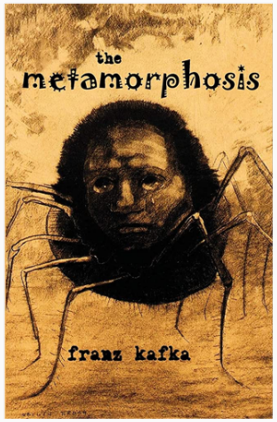
Virtue itself of vice must pardon beg, Yea, curb and woo for leave to do him good, She shall undo her credit with the judge, or own great place, Could fetch your brother from the angry law; do no stain to your own souls so blind That you will clear yourself from all suspense.

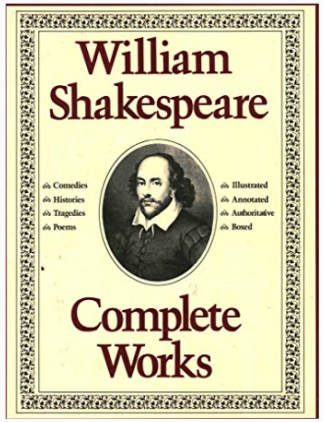
Lab 3

- Train model on free books from Project Gutenberg.
- Evaluate effect of changing k. Tradeoff between better performance and more "copying" from the source text.









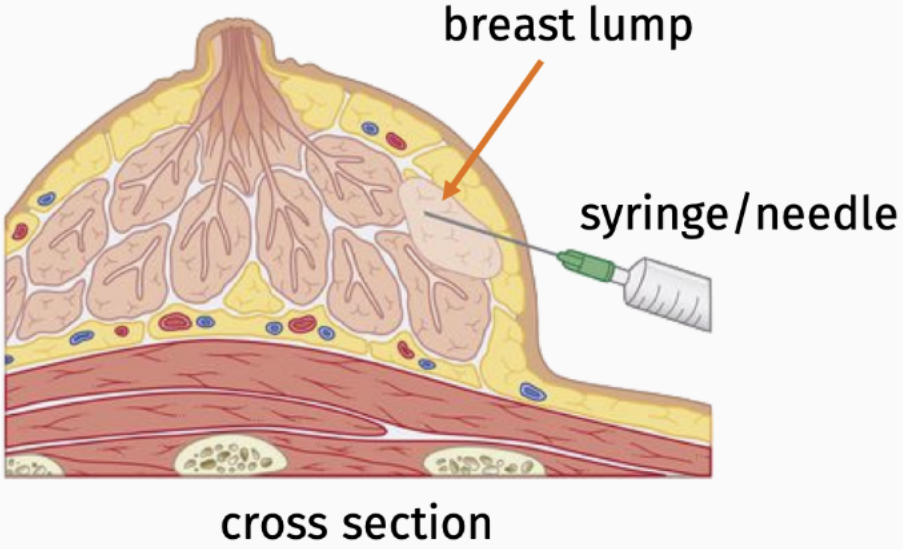
During this time, Madame Morrel had told her all,—'Giovanni,' said she, 'you should have brought this child with you; we would have replaced the parents it has lost, have called it Benedetto, and then, in a loyal duel, and not in Arabia, and in France eternal friendships are as rare as the custom of doing when saying "Yes." "Good; he accepts," said Monte Cristo.

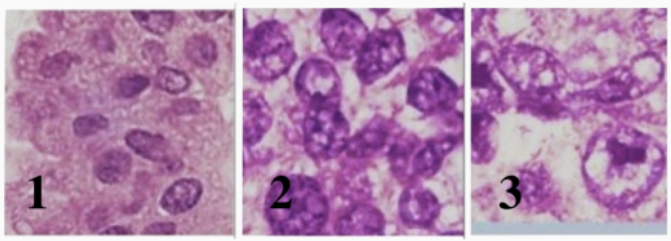
How to deal with non-binary cases?

Motivating problem

Breast Cancer Biopsy: Determine if a breast lump in a patient is malignant (cancerous) or benign (safe).

- Collect cells from lump using fine needle biopsy.
- Stain and examine cells under microscope.
- Based on certain characteristics (shape, size, cohesion) determine if likely malignant or not).





Motivating problem

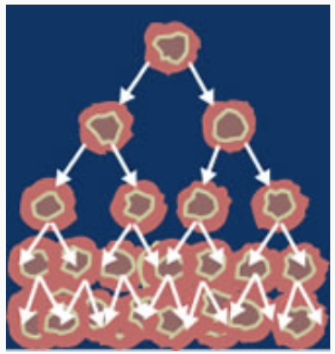
Demo: `demo_breast_cancer.ipynb`

Data: UCI machine learning repository

Breast Cancer Wisconsin (Original) Data Set

Download: Data Folder, Data Set Description

Abstract: Original Wisconsin Breast Cancer Database



Data Set Characteristics:	Multivariate	**Number of Instances:**	699	**Area:**	Life
Attribute Characteristics:	Integer	**Number of Attributes:**	10	**Date Donated**	1992-07-15
Associated Tasks:	Classification	**Missing Values?**	Yes	**Number of Web Hits:**	564320

`https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(original)`

Features: 10 numerical scores about cell characteristics (Clump Thickness, Uniformity, Marginal Adhesion, etc.)

Motivating problem

Data: $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$.

$\mathbf{x}_i = [1, 5, 4 \ldots, 2]$ contains score values.

Label $y_i \in \{0, 1\}$ is 0 if benign cells, 1 if malignant cells.

Goal: Based on scores (which would be collected manually, or even learned on their own using an ML algorithm) predict if a sample of cells is malignant or benign.

Approach:

- Naive Bayes Classifier can be extended to $\mathbf{x}$ with numerical values (instead of binary values as seen before). Will see on homework.

What are other classification algorithms people have heard of?

k-nearest neighbor method

k-NN algorithm: a simple but powerful baseline for classification.

Training data: $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$ where $y_1, \ldots, y_n \in \{1, \ldots, q\}$.

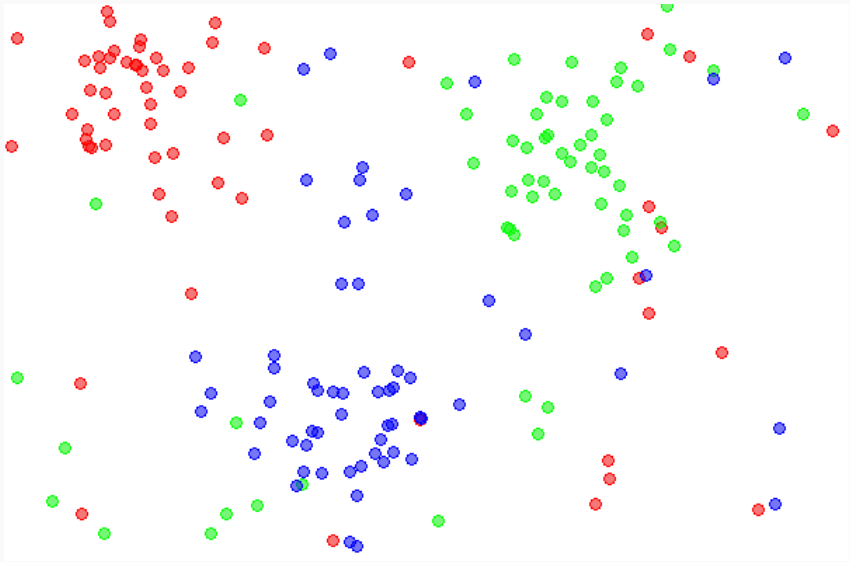
Classification algorithm:

Given new input $\mathbf{x}_{new}$,

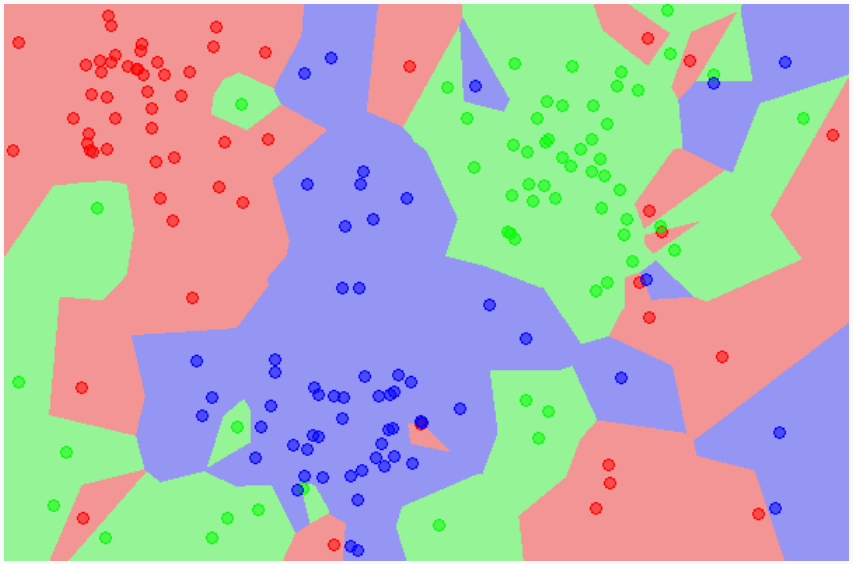
- Compute $sim(\mathbf{x}_{new}, \mathbf{x}_1), \ldots, sim(\mathbf{x}_{new}, \mathbf{x}_n)$.[2]
- Let $\mathbf{x}_{j_1}, \ldots, \mathbf{x}_{j_k}$ be the training data vectors with highest similarity to $\mathbf{x}_{new}$.
- Predict y_{new} as $majority(y_{j_1}, \ldots, y_{j_k})$.

[2] $sim(\mathbf{x}_{new}, \mathbf{x}_i)$ is any chosen similarity function, like $1 - \|\mathbf{x}_{new} - \mathbf{x}_i\|_2$.

k-nearest neighbor method



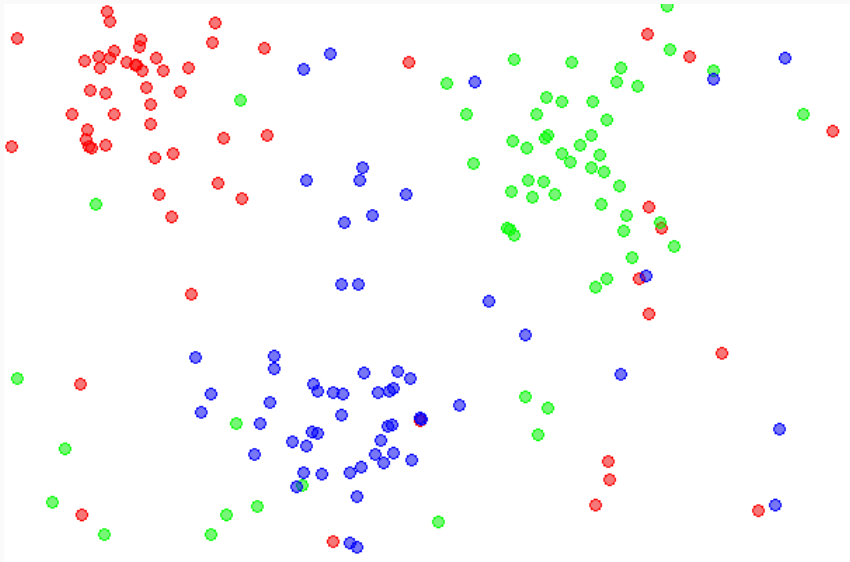
Data



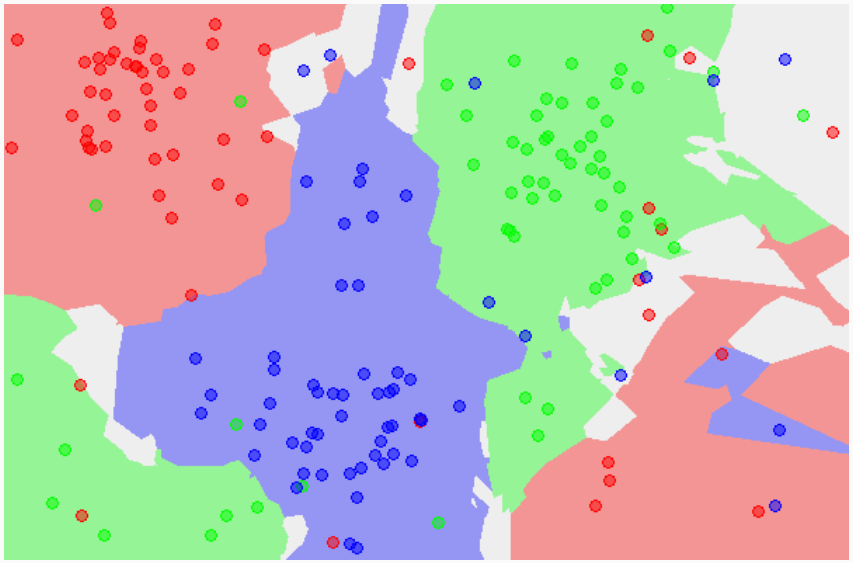
1-NN classifier

- Smaller k, more complex classification function.
- Larger k, more robust to noisy labels.

k-nearest neighbor method



Data



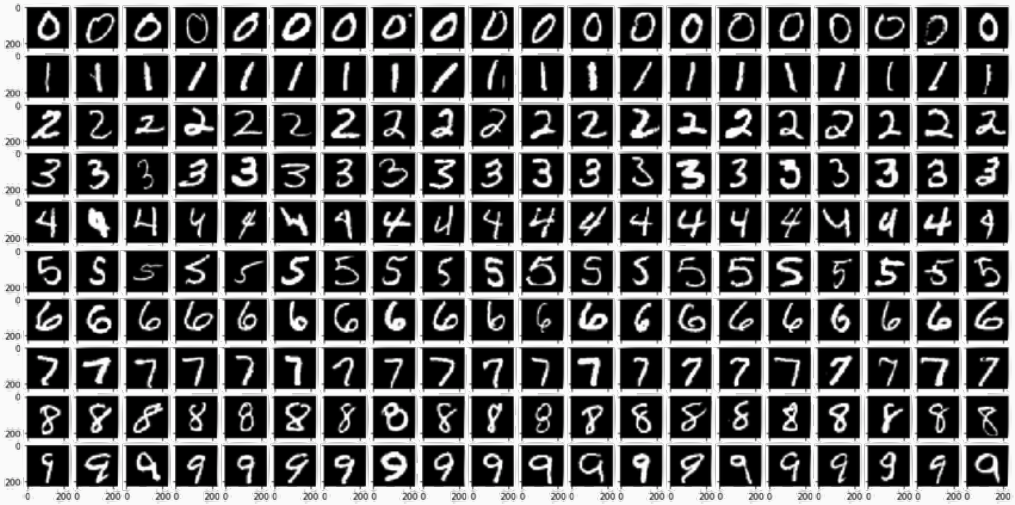
5-NN classifier

- Smaller k, more complex classification function.
- Larger k, more robust to noisy labels.
- Works remarkably well for many datasets.

MNIST image data

Especially good for large datasets with lots of repetition.

Example: works well on MNIST ($\approx 95\%$ accuracy out-of-the-box.):



Can be improved to 99.5% with a fancy similarity function![3]

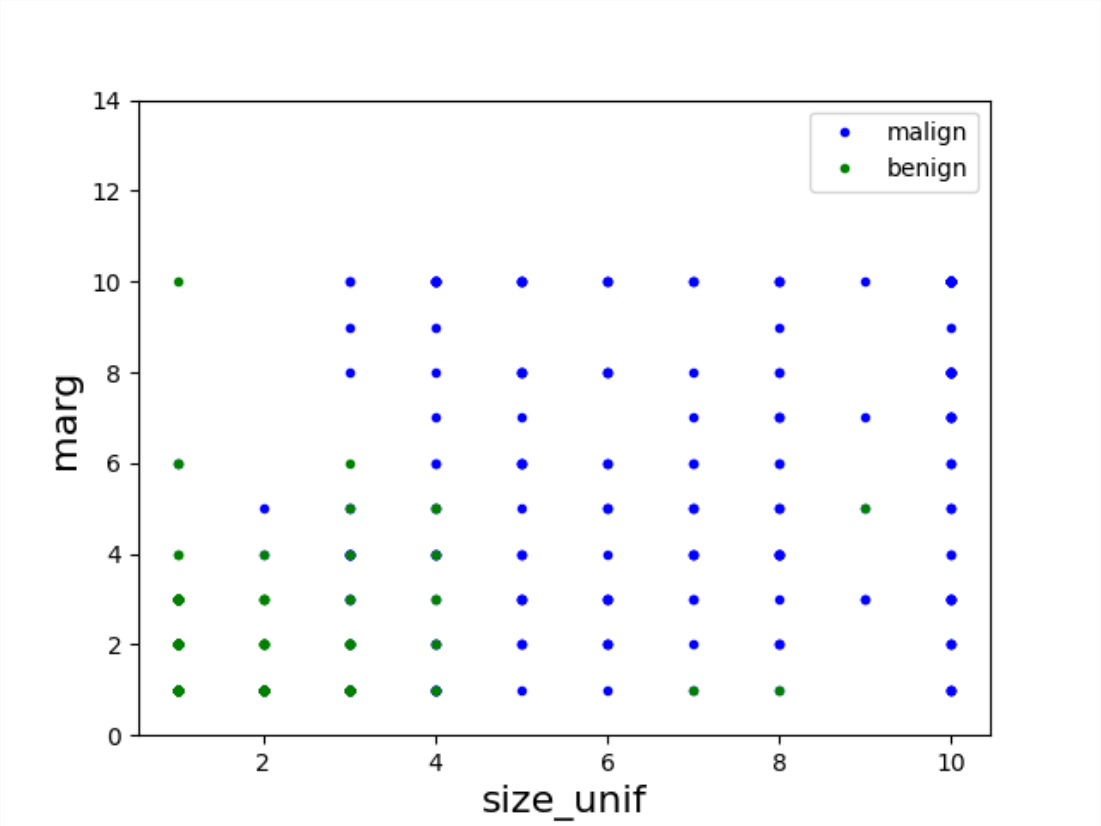
One issue is that prediction can be computationally intensive...

[3] We will revisit this when we talk about kernel methods.

Linear classification

Begin by plotting data

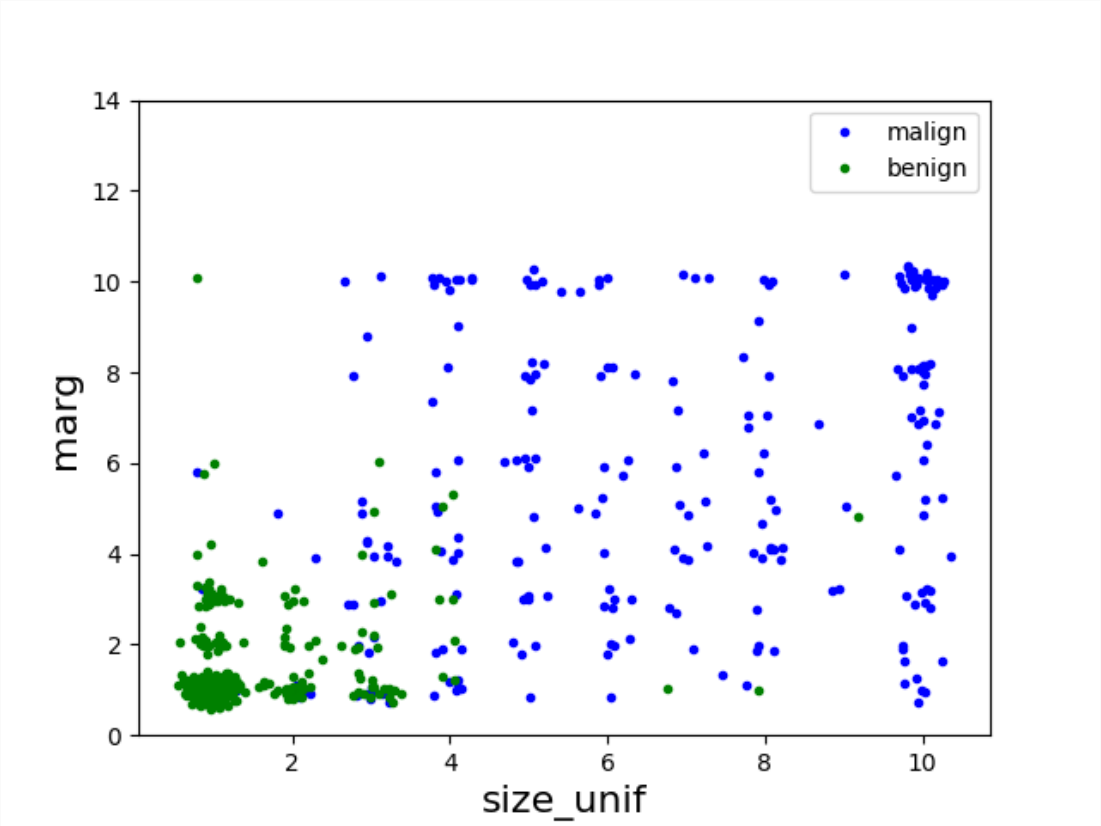
We pick two variables, Margin Adhesion and Size Uniformity and plot a scatter plot. Points with label 1 (malignant) are plotted in blue, those with label 2 (benign) are plotted in green.



Lots of overlapping points! Hard to get a sense of the data.

Plotting with jitter

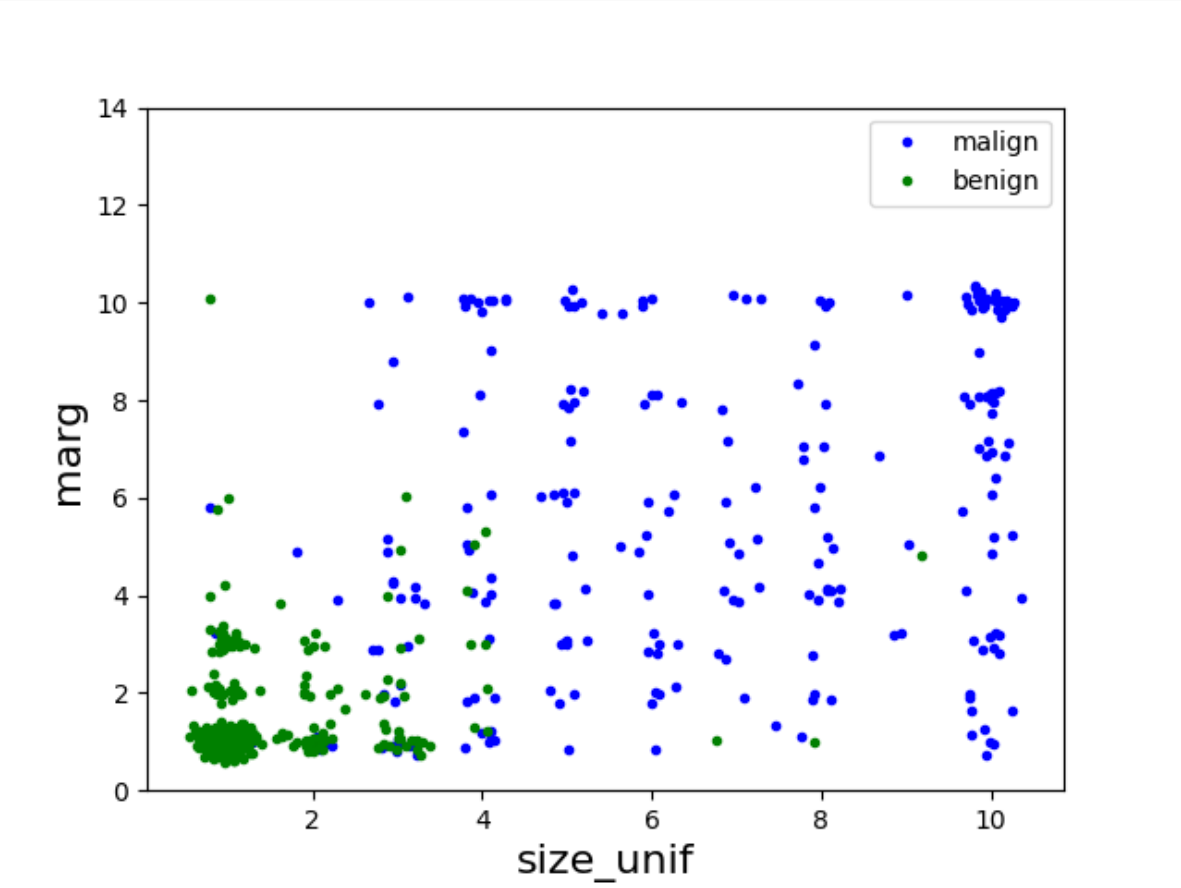
Simple + Useful Trick: data jittering. Add tiny random noise (using e.g. `np.random.randn`) to data to prevent overlap.



Noise is only for plotting. It is not added to the data for training, testing, etc.

Brainstorming

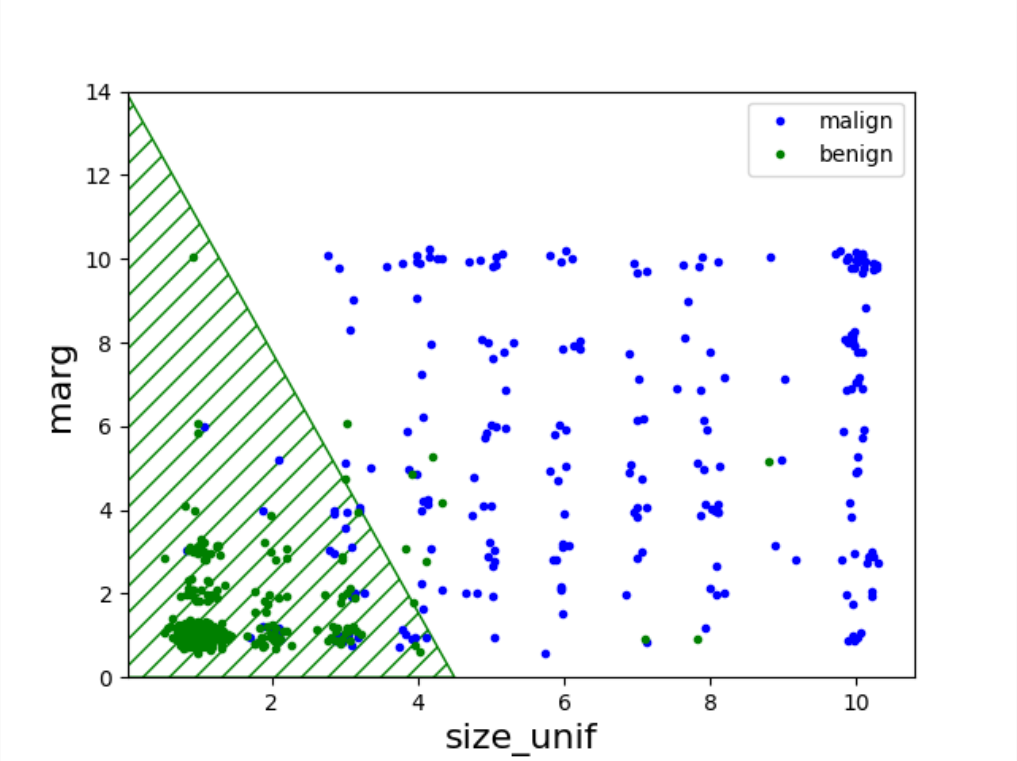
Any ideas for possible classification rules for this data?



Linear classifier

Given vector of predictors $\mathbf{x}_i \in \mathbb{R}^d$ (here $d = 2$) find a parameter vector $\boldsymbol{\beta} \in \mathbb{R}^d$ and threshold λ.

- Predict $y_i = 0$ if $\langle \mathbf{x}_i, \boldsymbol{\beta} \rangle \leq \lambda$.
- Predict $y_i = 1$ if $\langle \mathbf{x}_i, \boldsymbol{\beta} \rangle > \lambda$

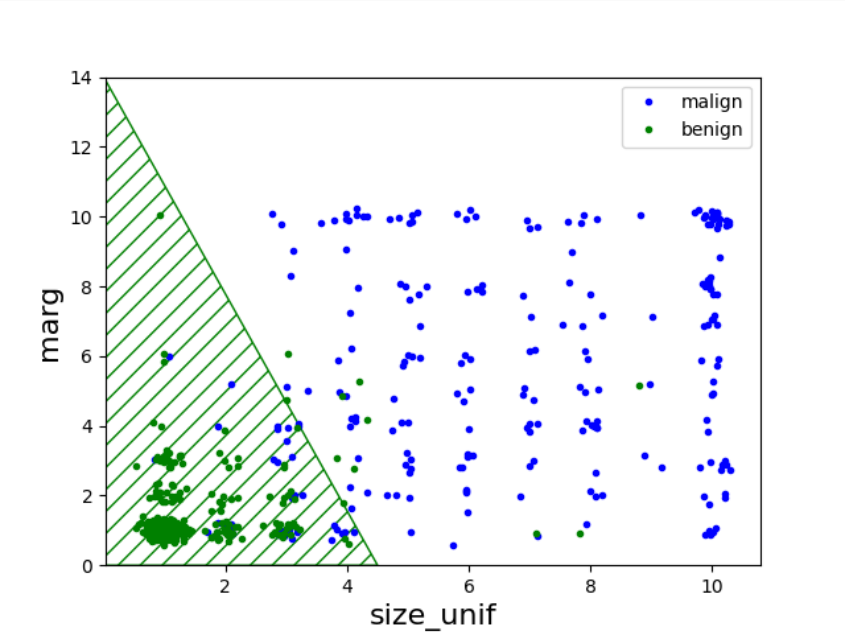


Line has equation $\langle \mathbf{x}, \boldsymbol{\beta} \rangle = \lambda$.

Linear classifier

As long as we append a 1 onto each data vector $\mathbf{x}_i$ (i.e. a column of ones onto the data matrix $\mathbf{X}$) like we did for linear regression, an equivalent function is:

- Predict $y_i = 0$ if $\langle \mathbf{x}_i, \boldsymbol{\beta} \rangle \leq 0$.
- Predict $y_i = 1$ if $\langle \mathbf{x}_i, \boldsymbol{\beta} \rangle > 0$

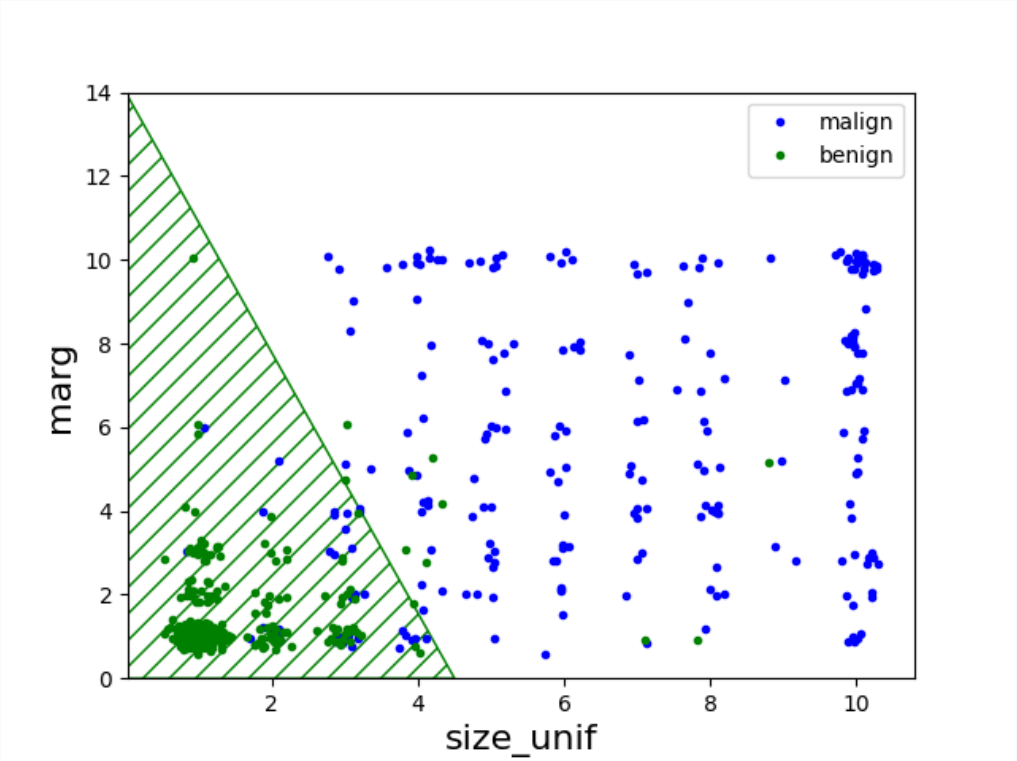


Line has equation $\langle \mathbf{x}, \boldsymbol{\beta} \rangle = 0$.

Linear classification

Standard approach for binary classification of real-valued data:

- Find parameter vector $\boldsymbol{\beta}$.
- For input data vector $\mathbf{x}$, predict 0 if $\boldsymbol{\beta}^T\mathbf{x} > \lambda$ and 1 $\boldsymbol{\beta}^T\mathbf{x} \leq \lambda$ for some threshold λ.[4]



[4] Can always assume $\lambda = 0$ if $\mathbf{x}$ has an intercept term.

0 - 1 loss

Question: How do we find a good linear classifier automatically?

Loss minimization approach (first attempt):

- **Model**[5]:

$$f_\beta(\mathbf{x}) = \mathbb{1}\left[\langle \mathbf{x}, \beta \rangle > 0\right]$$

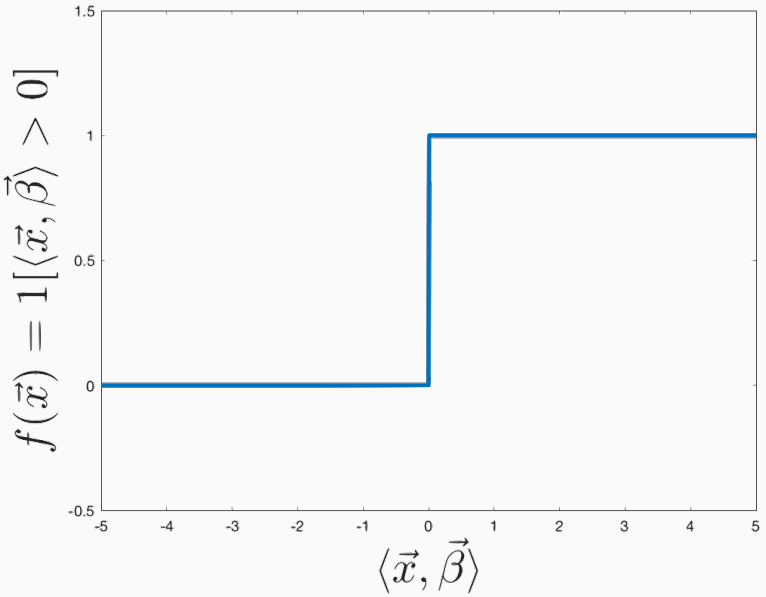
- **Loss function**: "$0-1$ Loss"

$$L(\beta) = \sum_{i=1}^{n} |f_\beta(\mathbf{x}_i) - y_i|$$

[5] $\mathbb{1}$[event] is the indicator function: it evaluates to 1 if the argument inside is true, 0 if false.

$0-1$ loss

Problem with $0-1$ loss:



- The loss function $L(\boldsymbol{\beta})$ is not differentiable because $f_{\boldsymbol{\beta}}(\mathbf{x})$ is discontinuous.
- Impossible to take the gradient, very hard to minimize loss to find optimal $\boldsymbol{\beta}$.
- Non-convex function (will make more sense next lecture).

Linear classifier via square loss

Loss minimization approach (second attempt):

- **Model**:

$$f_\beta(\mathbf{x}) = \mathbb{1}\left[\langle \mathbf{x}, \beta \rangle > 1/2\right]$$

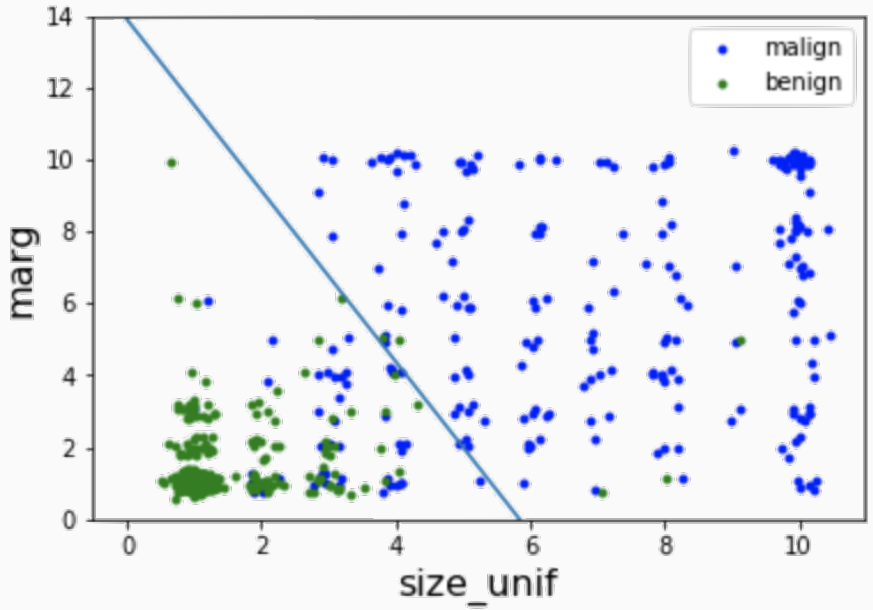
- **Loss function**: "Square Loss"

$$L(\beta) = \sum_{i=1}^{n} (\langle \mathbf{x}, \beta \rangle - y_i)^2$$

Intuitively tries to make $\langle \mathbf{x}, \beta \rangle$ close to 0 for examples in class 0, close to 1 for examples in class 1.

Linear classifier via square loss

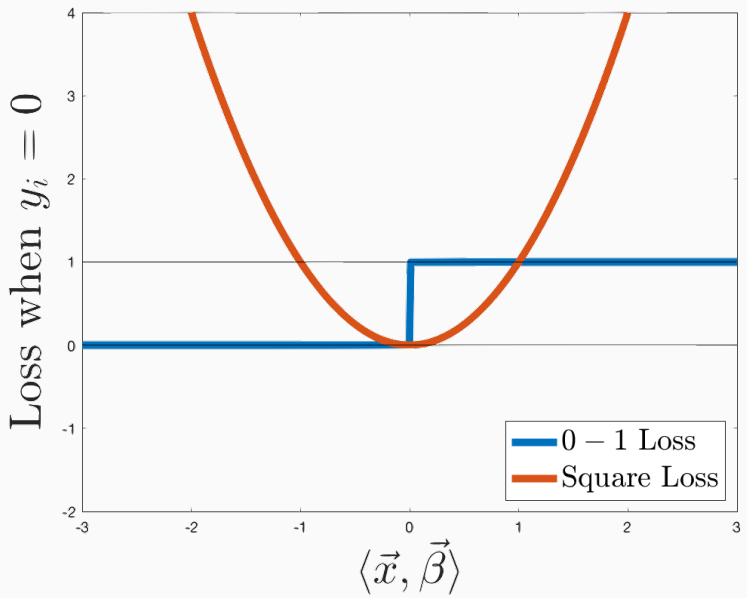
We can solve for β by just solving a least squares multiple linear regression problem.



Do you see any issues here?

Linear classifier via square loss

Problem with square loss:

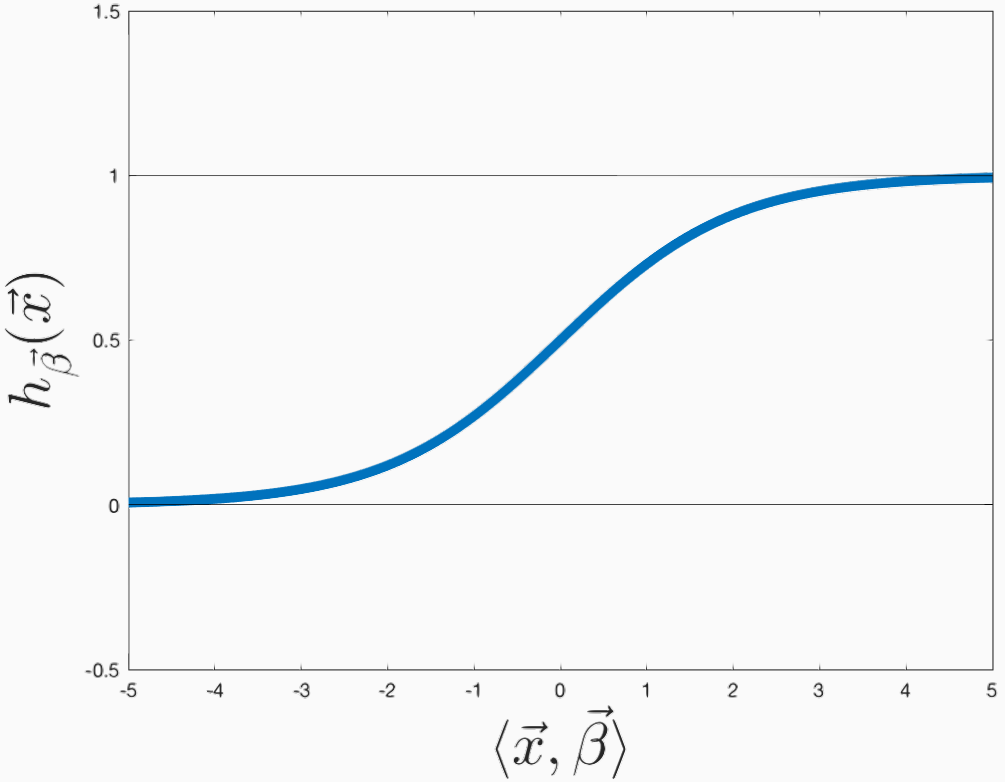


- Loss increases if $\langle \mathbf{x}, \boldsymbol{\beta} \rangle > 1$ even if correct label is 1. Or if $\langle \mathbf{x}, \boldsymbol{\beta} \rangle < 0$ even if correct label is 0.
- Intuitively we don't want to "punish" these cases.

Logistic regression

Let $h_{\boldsymbol{\beta}}(\mathbf{x})$ be the **logistic function**:

$$h_{\boldsymbol{\beta}}(\mathbf{x}) = \frac{1}{1 + e^{-\langle \boldsymbol{\beta}, \mathbf{x} \rangle}}$$



Logistic regression

Loss minimization approach (this works!):

- **Model**: Let $h_\beta(\mathbf{x}) = \frac{1}{1+e^{-\langle \beta, \mathbf{x} \rangle}}$

$$f_\beta(\mathbf{x}) = \mathbb{1}\left[h_\beta(\mathbf{x}) > 1/2\right]$$
$$= \mathbb{1}\left[\langle \mathbf{x}, \beta \rangle > 0\right]$$

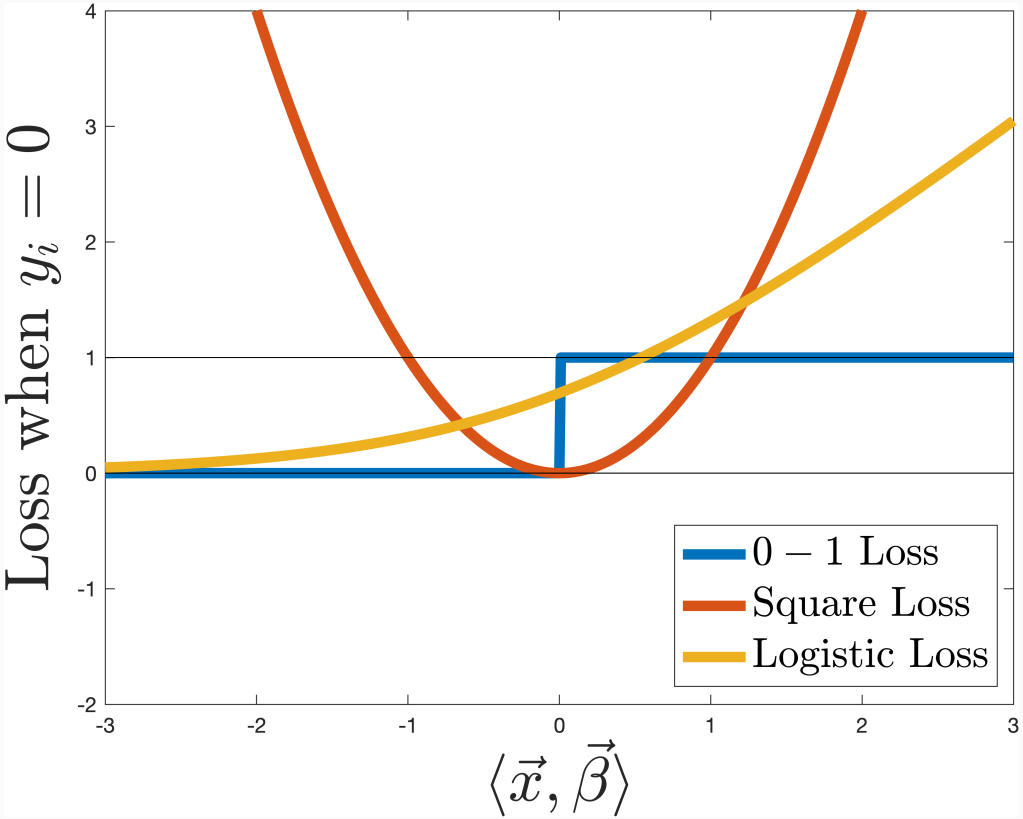
- **Loss function**: "Logistic loss" aka "binary cross-entropy loss"

$$L(\beta) = -\sum_{i=1}^{n} y_i \log(h_\beta(\mathbf{x})) + (1 - y_i) \log(1 - h_\beta(\mathbf{x}))$$

Logistic loss

Logistic Loss:

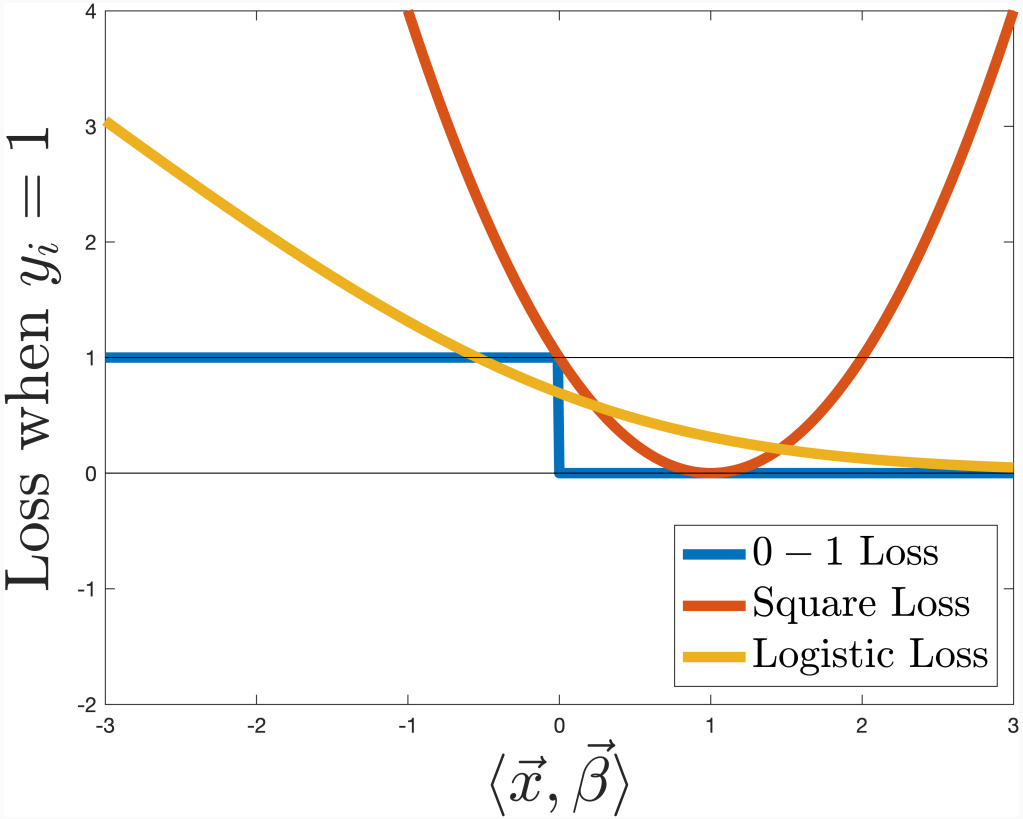
$$L(\beta) = -\sum_{i=1}^{n} y_i \log(h_\beta(\mathbf{x})) + (1 - y_i) \log(1 - h_\beta(\mathbf{x}))$$



Logistic loss

Logistic Loss:

$$L(\beta) = -\sum_{i=1}^{n} y_i \log(h_\beta(\mathbf{x})) + (1 - y_i) \log(1 - h_\beta(\mathbf{x}))$$



Logistic regression

Loss minimization approach:

- Given training data $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$.
- Minimize "Logistic loss" aka "binary cross-entropy loss"

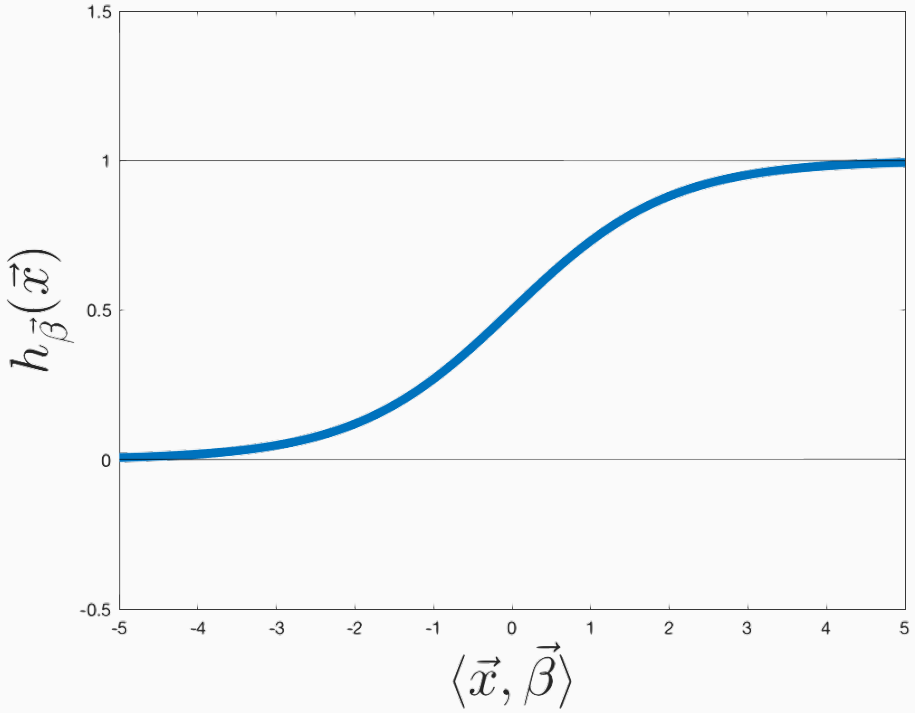
$$L(\beta) = -\sum_{i=1}^{n} y_i \log(h(\beta^T \mathbf{x}_i)) + (1 - y_i) \log(1 - h(\beta^T \mathbf{x}_i))$$

- Above $h(z)$ is the logistic/sigmoid function: $h(z) = \frac{1}{1+e^{-z}}$

Prediction: Predict $y_i = 1$ if $\beta^T \mathbf{x}_i \geq 0$, predict 0 otherwise.

Logistic regression

Let $h(z)$ be the logistic/sigmoid function: $h(z) = \frac{1}{1+e^{-z}}$



Can think of this function as mapping $\mathbf{x}^T\boldsymbol{\beta}$ to a probability that the true label is 1. If $\mathbf{x}^T\boldsymbol{\beta} \gg 0$ then the probability is close to 1, if $\mathbf{x}^T\boldsymbol{\beta} \ll 0$ then the probability is close to 0.

Exercise

Why not minimize:

$$L(\beta) = \sum_{i=1}^{n} \Big(y_i - h(\mathbf{x}^T \beta) \Big)^2 ?$$

Exercise

Why not minimize:

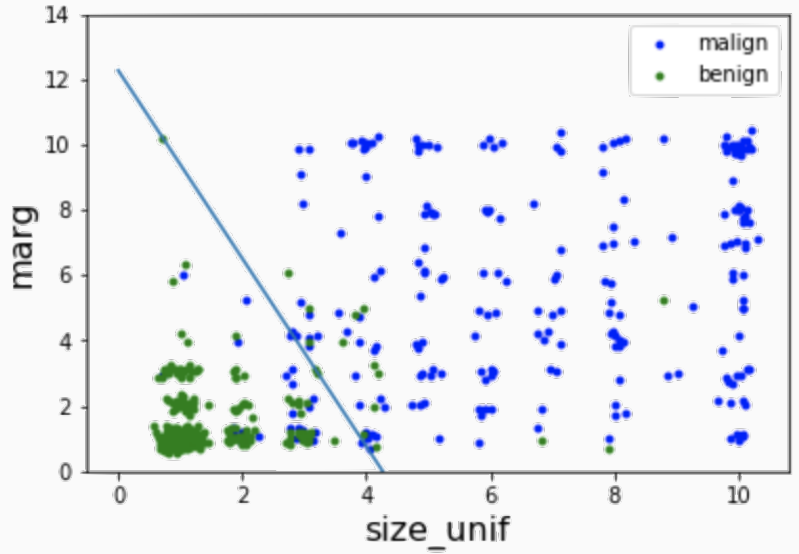
$$L(\beta) = \sum_{i=1}^{n} \Big(y_i - h(\mathbf{x}^T \beta) \Big)^2 ?$$

Answer: This is actually a pretty reasonable thing to do. An important issue however is that the loss here is not convex, which makes it hard to find the β that minimizes the loss.

Log-loss on the other hand is convex. More on this later.

Logistic loss

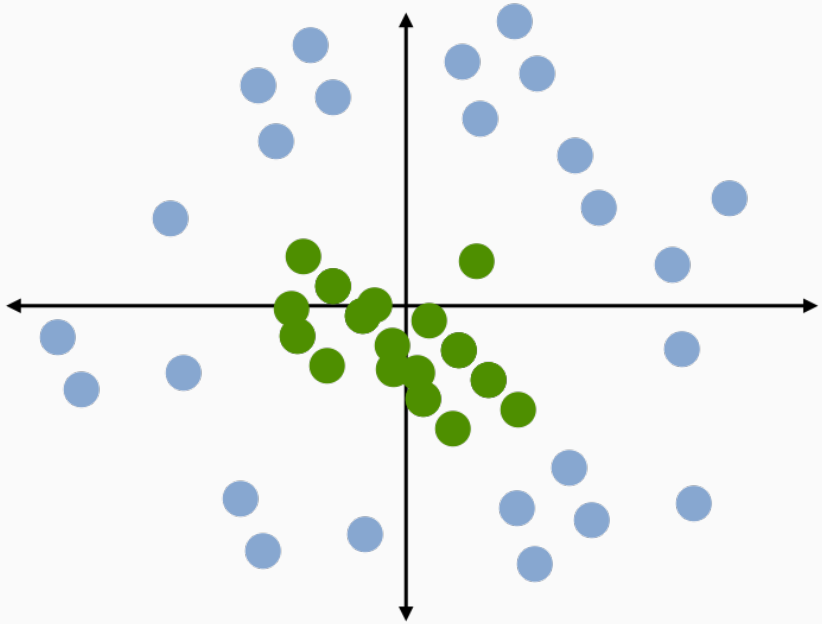
- Convex function in β, can be minimized using gradient descent.
- Works well in practice.
- Good Bayesian motivation.
- Easily combined with non-linear data transformations.



Fit using logistic regression/log loss.

Non-linear transformations

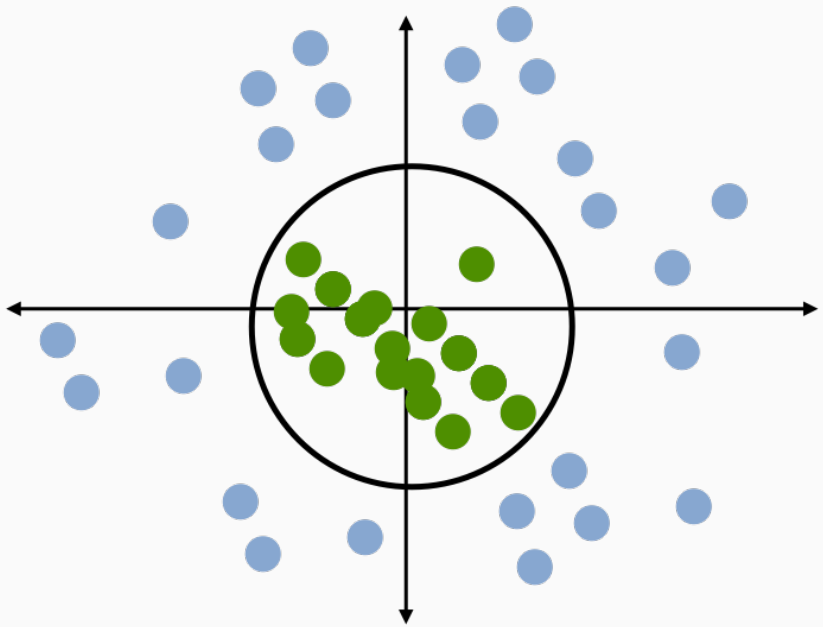
How would we learn a classifier for this data using logisitic regression?



This data is not linearly separable or even approximately linearly separable.

Non-linear transformations

Transform each $\mathbf{x} = [x_1, x_2]$ to $\mathbf{x} = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]$

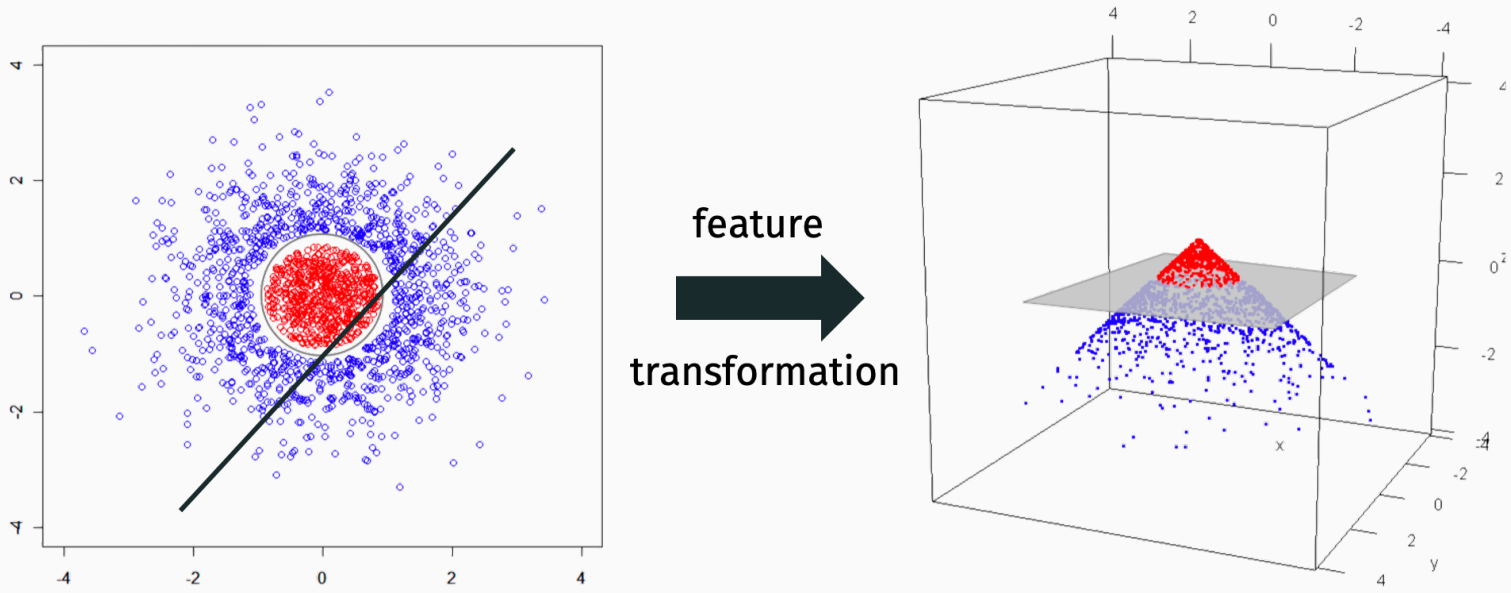


- Predict class 1 if $x_1^2 + x_2^2 < \lambda$.
- Predict class 0 if $x_1^2 + x_2^2 \geq \lambda$.

This is a linear classifier on our transformed data set. Logisitic regression might learn $\boldsymbol{\beta} = [0, 0, 0, 1, 1, 0]$.

Non-linear transformations

View as mapping data to a higher dimensional space, where it is linearly separable.



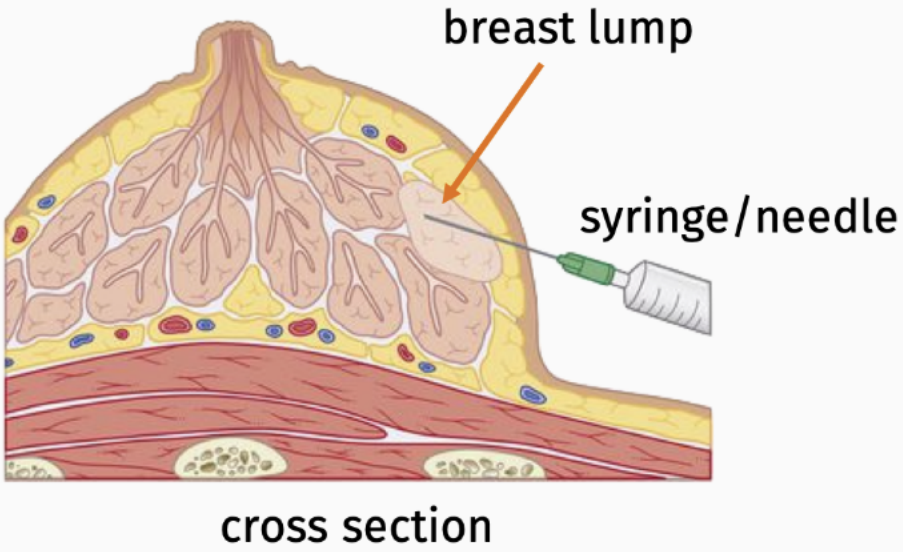
Lots more on this in future lecture!

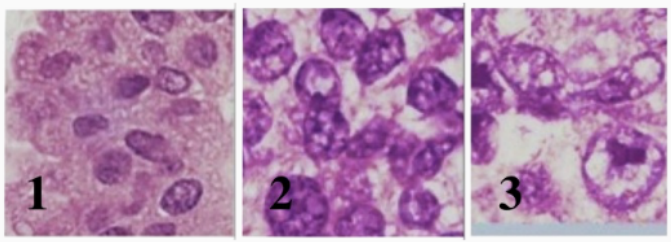
Error in classification

Once we have a classification algorithm, how do we judge its performance?

- **Simplest answer:** Error rate = fraction of data examples misclassified in test set.
- What are some issues with this approach?

Think back to motivating problem of breast cancer detection.

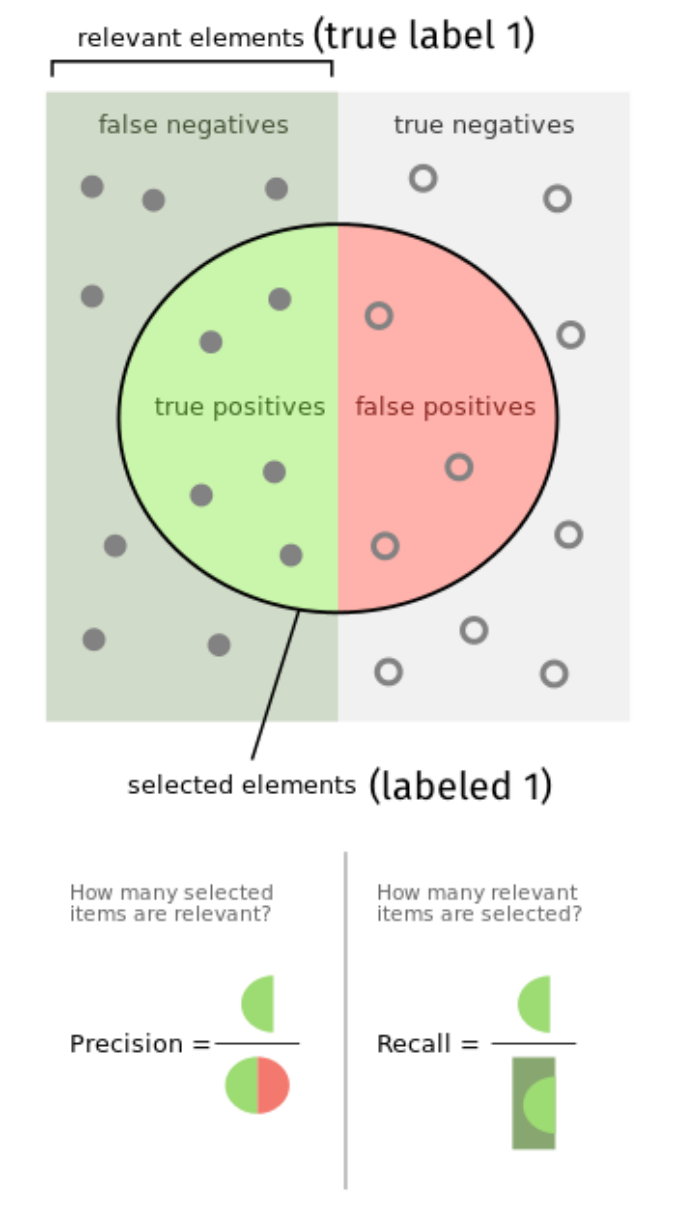




Error in classification

- **Precision:** Fraction of positively labeled examples (label 1) which are correct.
- **Recall:** Fraction of true positives that we labeled correctly with label 1.

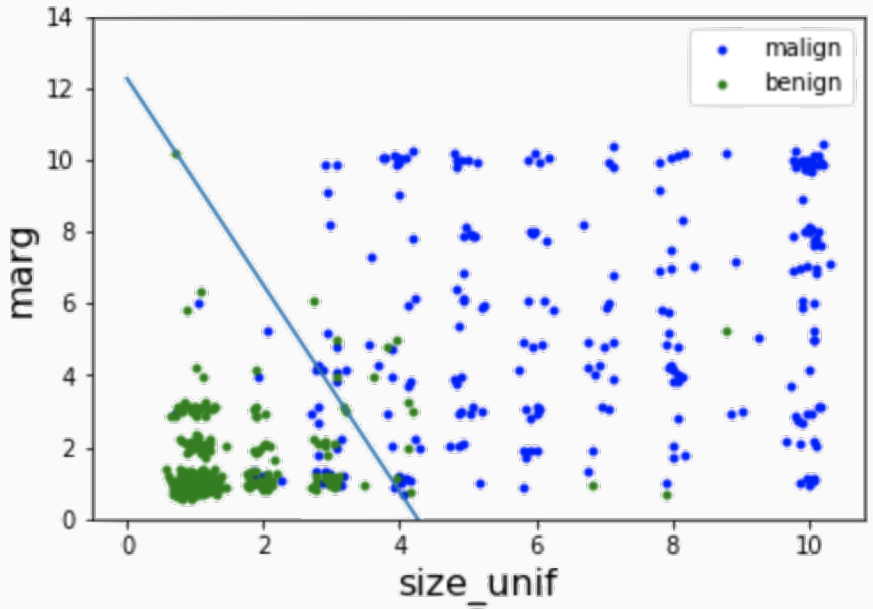
Question: Which should we optimize for medical diagnosis?



Error in classification

Possible logistic regression workflow:

- Learn $\vec{\beta}$ and compute $h_{\vec{\beta}}(\vec{x}_i) = \frac{1}{1+e^{-\langle \vec{x}_i, \vec{\beta} \rangle}}$ for all $\vec{x}_i$.
- Predict $y_i = 0$ if $h_{\vec{\beta}}(\vec{x}_i) \leq \lambda$, $y_i = 1$ if $h_{\vec{\beta}}(\vec{x}_i) > \lambda$.
- Default value of λ is $1/2$. Increasing λ improves precision. Decreasing λ improves recall.



Error in classification

Possible logistic regression workflow:

- Learn $\vec{\beta}$ and compute $h_{\vec{\beta}}(\vec{x}_i) = \frac{1}{1+e^{-\langle \vec{x}_i, \vec{\beta} \rangle}}$ for all $\vec{x}_i$.
- Predict $y_i = 0$ if $h_{\vec{\beta}}(\vec{x}_i) \leq \lambda$, $y_i = 1$ if $h_{\vec{\beta}}(\vec{x}_i) > \lambda$.
- Default value of λ is $1/2$. Increasing λ improves precision. Decreasing λ improves recall.

This is very heuristic. There are other methods for handling "class imbalance" which can often lead to good overall error, but poor precision or recall. Techniques include weighting the loss function to care more about false negatives, or subsampling the larger class.

Multi-class

What about when $y \in \{1, \ldots, q\}$ instead of $y \in \{0, 1\}$

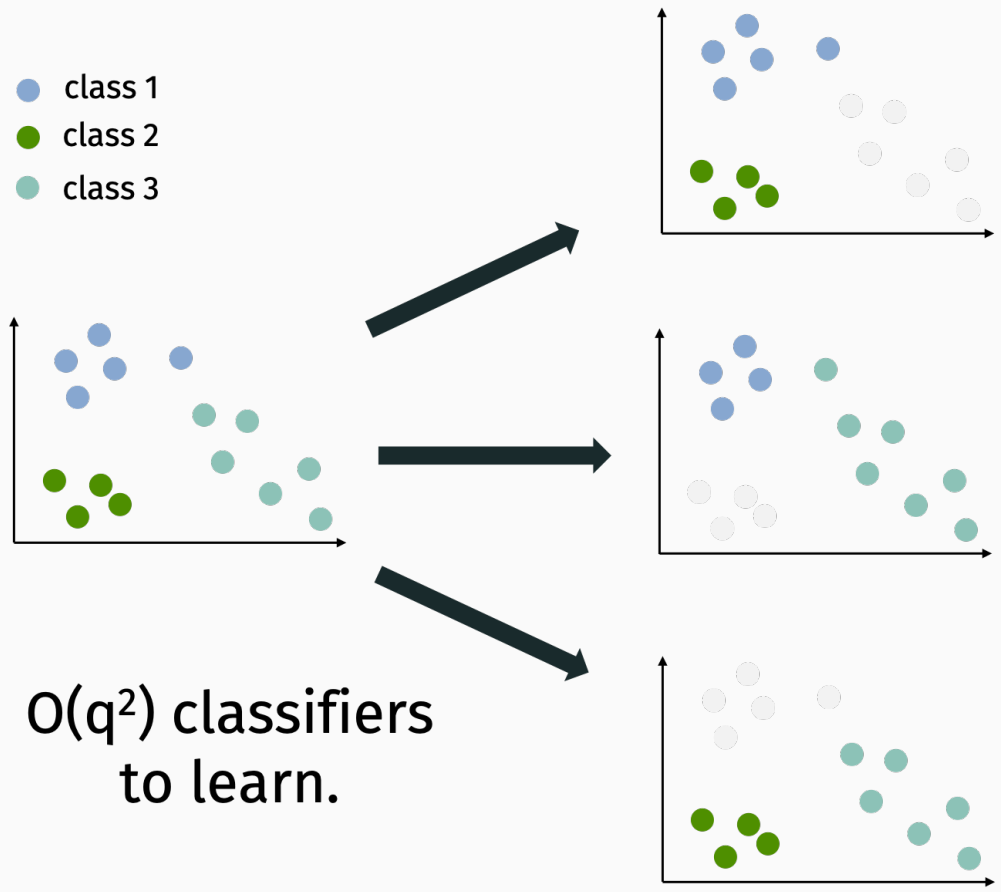
Two common options for reducing multi-class problems to binary problems:

- One-vs.-all (most common, also called one-vs.-rest)
- One-vs.-one (slower, but can be more effective)

One vs. rest

- For q classes train q classifiers. Obtain parameters $\beta^{(1)}, \ldots, \beta^{(q)}$.
- Assign y to class i if $\langle \beta^{(i)}, \mathbf{x} \rangle \geq 0$. Could be ambiguous!
- **Better:** Assign y to class i with maximum value of $h(\langle \beta^{(i)}, \mathbf{x} \rangle)$.

One vs. one



- For q classes train $\frac{q(q-1)}{2}$ classifiers.
- Assign y to class which i which wins in the most number of head-to-head comparisons.

One vs. one

Hard case for one-vs.-all.

- One-vs.-one would be a better choice here.
- Also tends to work better when there is class in balance.
- But one-vs.-one can be super expensive! E.g when $q = 100$ or $q = 1000$.

Multiclass logistic regression

More common modern alternative: If we have q classes, train a single model with q parameter vectors $\beta^{(1)}, \ldots, \beta^{(q)}$, and predict class $i = \arg\max_i \langle \beta^{(i)}, \mathbf{x} \rangle$.

Same idea as one-vs.-rest, but we treat $[\beta^{(1)}, \ldots, \beta^{(q)}]$ as a single length qd parameter vector which we to optimize to minimize a single joint loss function. We do not train the parameter vectors separately.

What's a good loss function?

Multiclass logistic regression

Softmax function:

$$\begin{bmatrix} \langle \boldsymbol{\beta}^{(1)}, \mathbf{x} \rangle \\ \vdots \\ \langle \boldsymbol{\beta}^{(q)}, \mathbf{x} \rangle \end{bmatrix} \xrightarrow{\text{softmax}} \begin{bmatrix} e^{\langle \boldsymbol{\beta}^{(1)}, \mathbf{x} \rangle} / \sum_{i=1}^{q} e^{\langle \boldsymbol{\beta}^{(i)}, \mathbf{x} \rangle} \\ \vdots \\ e^{\langle \boldsymbol{\beta}^{(q)}, \mathbf{x} \rangle} / \sum_{i=1}^{q} e^{\langle \boldsymbol{\beta}^{(i)}, \mathbf{x} \rangle} \end{bmatrix}$$

Softmax takes in a vector of numbers and converts it to a vector of probabilities:

$$\begin{bmatrix} -10 & 4 & 1 & 0 & -5 \end{bmatrix} \to \begin{bmatrix} .00 & .94 & .04 & .02 & -.00 \end{bmatrix}$$

Multiclass logistic regression

Multi-class cross-entropy:

$$L(\beta^{(1)}, \ldots, \beta^{(q)}) = -\sum_{i:y_i=1} \log \frac{e^{\langle \beta^{(1)}, \mathbf{x}_i \rangle}}{\sum_{j=1}^{q} e^{\langle \beta^{(j)}, \mathbf{x}_i \rangle}} - \ldots - \sum_{i:y_i=q} \log \frac{e^{\langle \beta^{(q}, \mathbf{x}_i \rangle}}{\sum_{j=1}^{q} e^{\langle \beta^{(j)}, \mathbf{x}_i \rangle}}$$

$$= -\sum_{i=1}^{n} \sum_{\ell=1}^{q} \mathbb{1}[y_i = \ell] \cdot \log \frac{e^{\langle \beta^{(\ell)}, \mathbf{x}_i \rangle}}{\sum_{j=1}^{q} e^{\langle \beta^{(j)}, \mathbf{x}_i \rangle}}$$

Binary cross-entropy:

$$L(\beta) = -\sum_{i=1}^{n} y_i \log(h(\beta^T \mathbf{x}_i)) + (1 - y_i) \log(1 - h(\beta^T \mathbf{x}_i))$$

$$= -\sum_{i:y_i=1} \log(h(\beta^T \mathbf{x}_i)) - \sum_{i:y_i=0} \log(1 - h(\beta^T \mathbf{x}_i))$$

Not exactly the same... but can show equivalent if you set $\beta^{(0)} = \beta$ and $\beta^{(1)} = -\beta$.

Error in (multiclass) classification

Confusion matrix for k classes:

Pred--> Real↓	1	2	...	K
1				
2				
...				
K				

- Entry i, j is the fraction of class i items classified as class j.
- Useful to see whole matrix to visualize where errors occur.